HOSPITAL MANAGEMENT SYSTEM

Database Report

Schema, relationships, normalization, views, triggers, procedures, queries, transactions, integrity and security of `hospital_db`

Database	hospital_db on MySQL 9.7.1, at migration 014_patient_dob_estimated
Code	branch main, commit 0c32974
Date	12 October 2026
Source of truth	The live schema, read through information_schema and SHOW CREATE TABLE / VIEW / TRIGGER / PROCEDURE, and the migration files. Not the old database/schema.sql, which does not match it.

Contents

1. Overview 4
- 1.1 Purpose of the system 4
- 1.2 Server and schema facts 4
- 1.3 Object counts 4

2. ER diagrams 6
- 2.1 Full diagram: all 32 tables and 73 relationships 7
- 2.2 Module: Patients and visits 8
- 2.3 Module: Beds, admissions and emergency 9
- 2.4 Module: Pharmacy 10
- 2.5 Module: Laboratory 11
- 2.6 Module: Staff, billing and audit 12

3. Relationships 13

4. Table schemas 18

5. Functional dependencies and candidate keys 44

6. Normalization 46
- 6.1 Method 46
- 6.2 Verdict per table 46
- 6.3 Deliberate denormalization 48
- 6.4 Worked example: from an unnormalized record to these tables 50

7. Views 52
- 7.1 available_beds_view 52
- 7.2 available_doctors_view 52
- 7.3 bed_status_summary_view 52
- 7.4 current_admissions_view 53
- 7.5 doctor_workload_view 53
- 7.6 emergency_queue_view 53
- 7.7 hospital_occupancy_view 54
- 7.8 pending_bills_view 54
- 7.9 v_current_stock 54

8. Triggers 56
- 8.1 after_admission_discharge 56
- 8.2 after_admission_insert 56
- 8.3 audit_logs_no_delete 56
- 8.4 audit_logs_no_update 57
- 8.5 after_bed_assignment_log 57
- 8.6 before_medicine_stock_update 57
- 8.7 after_prescription_dispense 57

9. Stored procedures and functions 59
- 9.1 sp_admit_patient 59
- 9.2 sp_allocate_emergency 59
- 9.3 sp_discharge_patient 59
- 9.4 sp_generate_bill 59
- 9.5 sp_get_emergency_queue 60

10. Complex SQL queries 61
- 10.1 Is this patient in the doctor's care set? (access control) 61
- 10.2 A patient's prescriptions, each with its labelled medicine lines 61
- 10.3 One day's appointments with patient, doctor and visit status 62

10.4 Visits left open from earlier days 62
10.5 A nurse's wards: every bed, and its patient if occupied 63
10.6 Open outpatient visits with the time vitals were last taken 63
10.7 Vital-sign readings with where each was filed 64
10.8 Medicine stock picture (uses the view v_current_stock) 64
10.9 Bed counts by type and status 65
10.10 Medicines whose stored total disagrees with their batches (invariant check) 65
10.11 Patients with more than one active admission (anomaly check) 65
10.12 Departments ranked by number of admissions 65
10.13 Distinct patients a doctor saw today (dashboard) 66
10.14 Dispensing: lock the prescription, then medicines in id order, then batches first-expiry-first 66
10.15 Bedside dose: lock one scheduled dose with its order, prescription and medicine 67

11. Transactions and concurrency 68

12. Integrity and security 70

13. Evolution 72

14. Viva questions 73

Section 4 has one subsection per table (4.1–4.33), listed on the next lines.

4.1 `patients` 18
4.2 `appointments` 19
4.3 `encounters` 19
4.4 `consultations` 21
4.5 `vital_signs` 22
4.6 `surgery_requests` 23
4.7 `departments` 23
4.8 `wards` 24
4.9 `beds` 24
4.10 `admissions` 25
4.11 `bed_assignment_log` 26
4.12 `emergency_cases` 27
4.13 `ambulances` 28
4.14 `medicines` 28
4.15 `medicine_batches` 29
4.16 `prescriptions` 30
4.17 `prescription_items` 31
4.18 `prescription_item_batches` 32
4.19 `pharmacy_stock_movements` 32
4.20 `medication_administrations` 33
4.21 `lab_tests` 34
4.22 `lab_orders` 34
4.23 `lab_results` 35
4.24 `lab_result_attachments` 36
4.25 `users` 37
4.26 `doctors` 37
4.27 `doctor_schedules` 38
4.28 `doctor_daily_analytics` 39
4.29 `nurse_ward_assignments` 39
4.30 `bills` 40
4.31 `bill_items` 41
4.32 `audit_logs` 41
4.33 `schema_migrations` 42

1. Overview

1.1 Purpose of the system

The Hospital Management System (HMS) runs the day-to-day work of a hospital:

- registering patients and booking appointments;
- the outpatient visit, from arrival through triage to the doctor;
- clinical notes, vital signs, lab orders and results, prescriptions and dispensing, and the bedside medication record;
- admissions, beds and wards, the emergency queue and ambulances;
- billing, staff accounts with role-based access, and an audit trail of who did what.

It is a Node.js/Express API over MySQL, with a React client. This report describes the database: the live schema of `hospital_db` (the development database), read through `information_schema` and `SHOW CREATE`, together with the migration files in `database/migrations`. The older files `database/schema.sql` and `lab_pharmacy_workflow.sql` do not match the live database and were not used.

1.2 Server and schema facts

Database	`hospital_db` (development database on the project laptop), at migration `014_patient_dob_estimated`. All 15 migration files in database/migrations are applied.
MySQL version	9.7.1
Storage engine	InnoDB for every table (server default InnoDB)
Isolation level	REPEATABLE-READ (InnoDB default; the application does not change it)
Character set / collation	utf8mb4 / utf8mb4_0900_ai_ci
SQL mode	`ONLY_FULL_GROUP_BY, STRICT_TRANS_TABLES, NO_ZERO_IN_DATE, NO_ZERO_DATE, ERROR_FOR_DIVISION_BY_ZERO, NO_ENGINE_SUBSTITUTION`
Binary log	on; InnoDB lock wait timeout 50 s
Samples	The sample results in section 10 come from `hospital_db_test`, which has the same schema (schema-diff: no differences) and is filled with test data.

1.3 Object counts

Tables	33 (32 for the hospital, plus schema_migrations)
Views	9
Triggers	7
Stored procedures	5
Stored functions	0 (none)
Events	0
Columns	344
Indexes	126 (33 primary, 16 unique, 77 non-unique)
Foreign keys	74 (ON DELETE: 54 RESTRICT, 12 CASCADE, 8 SET NULL; ON UPDATE: all NO ACTION)
CHECK constraints	29

Tables by module, with their row counts in hospital_db (development data):

Module	Tables (rows)
Patients and visits	`appointments` (9), `consultations` (3), `encounters` (9), `patients` (36), `surgery_requests` (0), `vital_signs` (2)
Beds, admissions and emergency	`admissions` (19), `ambulances` (6), `bed_assignment_log` (19), `beds` (35), `departments` (9), `emergency_cases` (9), `wards` (6)
Pharmacy	`medication_administrations` (15), `medicine_batches` (22), `medicines` (22), `pharmacy_stock_movements` (6), `prescription_item_batches` (4), `prescription_items` (13), `prescriptions` (4)
Laboratory	`lab_orders` (9), `lab_result_attachments` (1), `lab_results` (6), `lab_tests` (15)
Staff, billing and audit	`audit_logs` (3552), `bill_items` (19), `bills` (4), `doctor_daily_analytics` (4), `doctor_schedules` (36), `doctors` (12), `nurse_ward_assignments` (2), `schema_migrations` (15), `users` (17)

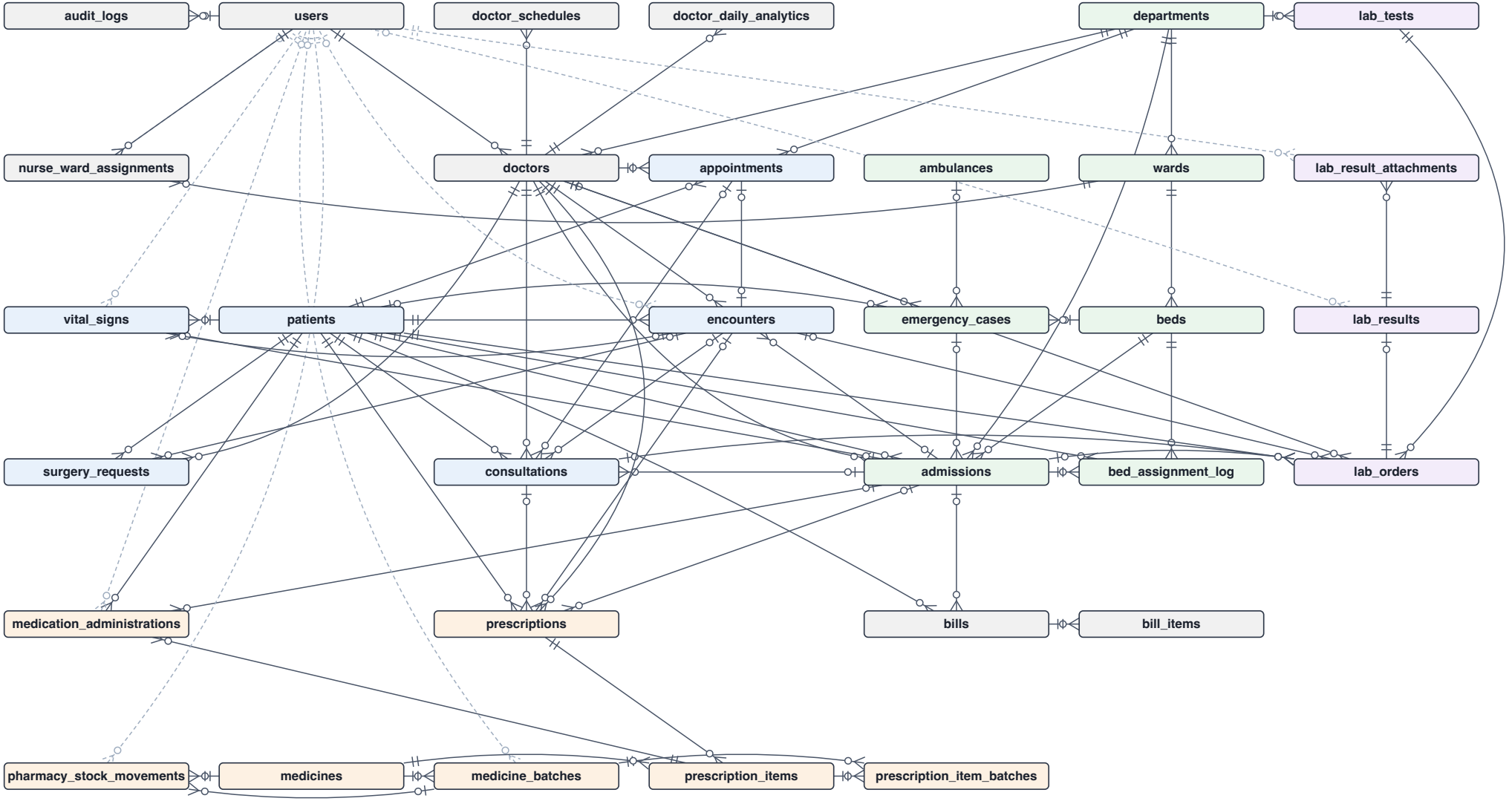
2. ER diagrams

How to read the diagrams. One box per table, coloured by module. One line per foreign key, in crow's-foot notation:

- **At the referenced (parent) table:** || means exactly one; o| means zero or one (the foreign key column allows NULL).
- **At the referencing (child) table:** a crow's foot with a circle means zero or many; a bar with a circle means zero or one (the foreign key column is unique).
- **Light dashed lines** are "who recorded / created / performed" links to `users`.

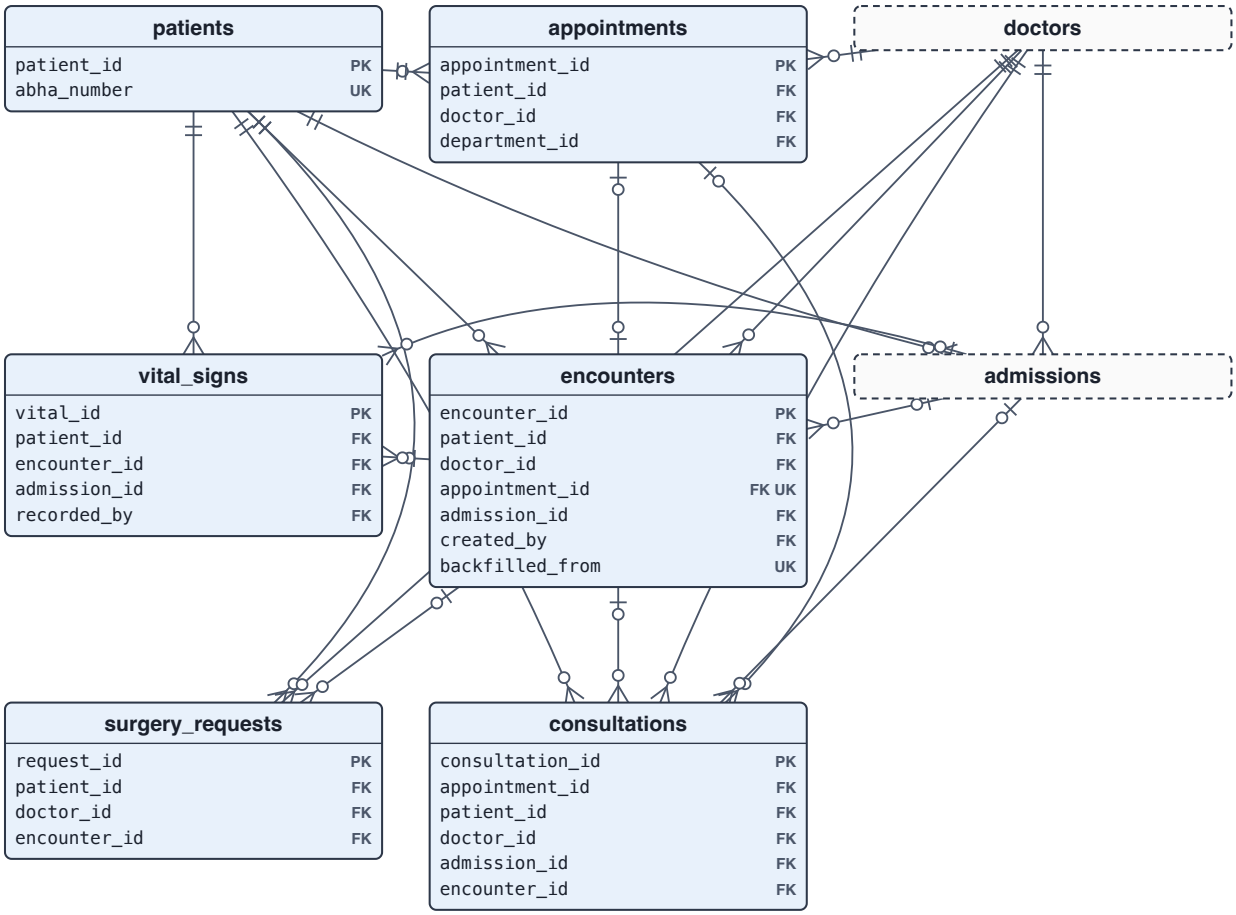
Lines curve around boxes, so a line always ends at the table it belongs to. Section 2.1 shows every table and relationship. Because it is dense, sections 2.2–2.6 repeat each module larger, with its key columns (PK, FK, UK). There, tables from other modules appear as plain grey context boxes.

2.1 Full diagram: all 32 tables and 73 relationships



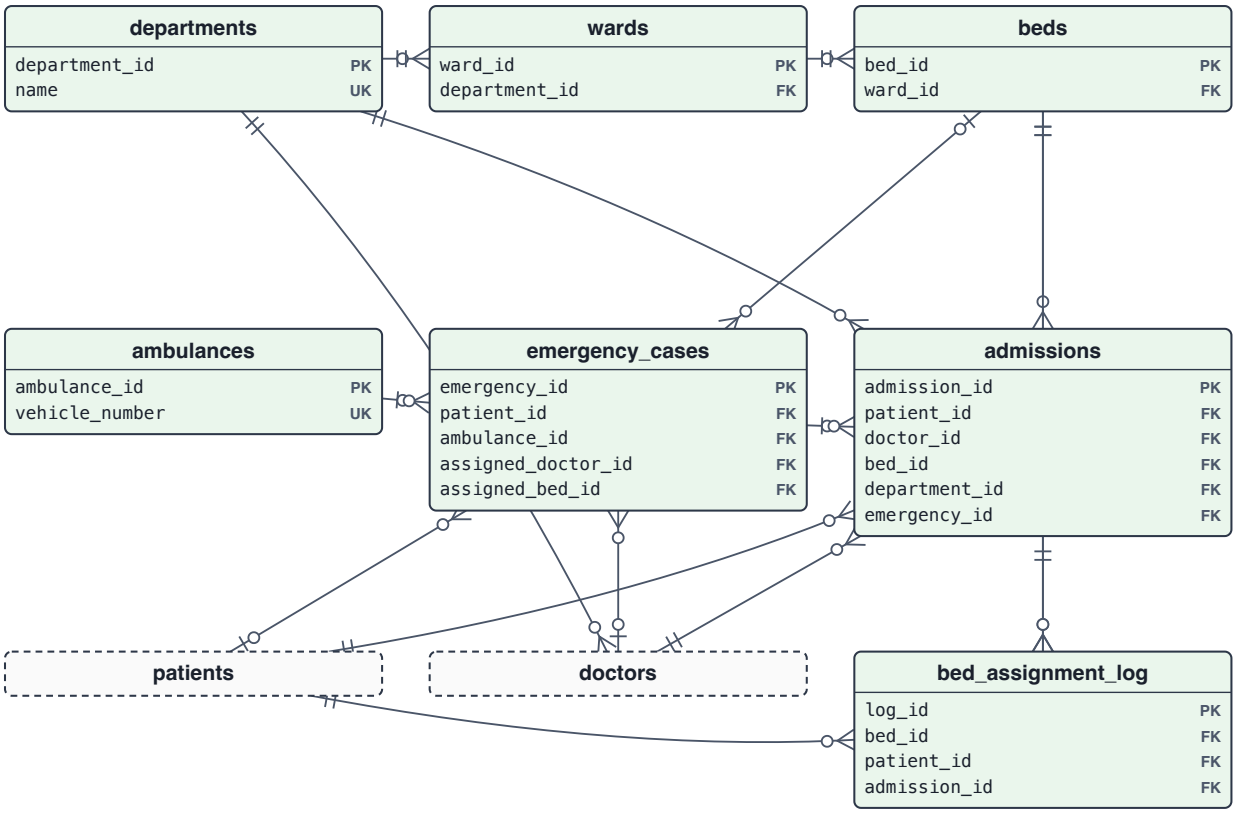
Not shown: `schema_migrations` (no relationships). Colours: blue patients and visits, green beds and admissions, orange pharmacy, purple laboratory, grey staff, billing and audit.

2.2 Module: Patients and visits



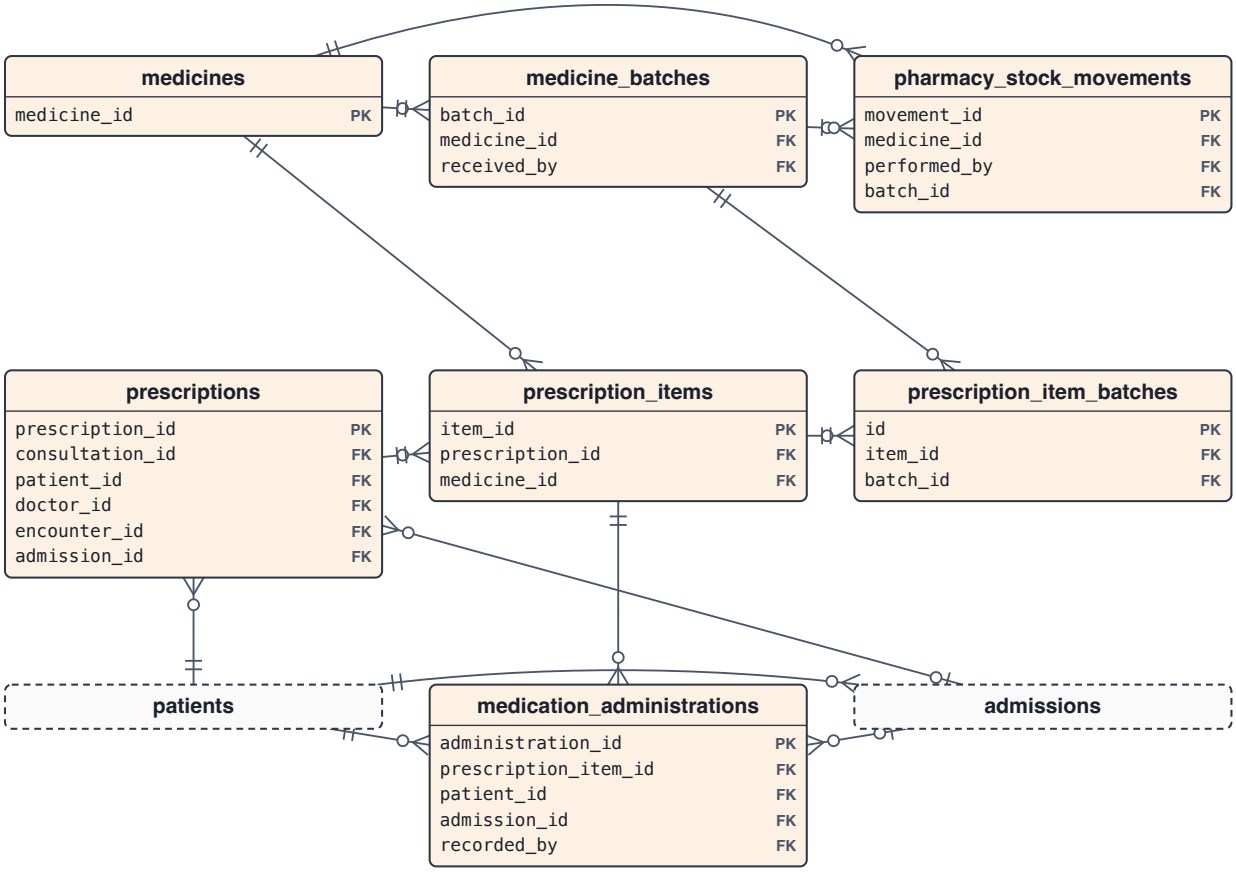
19 relationships among these tables; links to tables outside the figure are listed in section 3.

2.3 Module: Beds, admissions and emergency



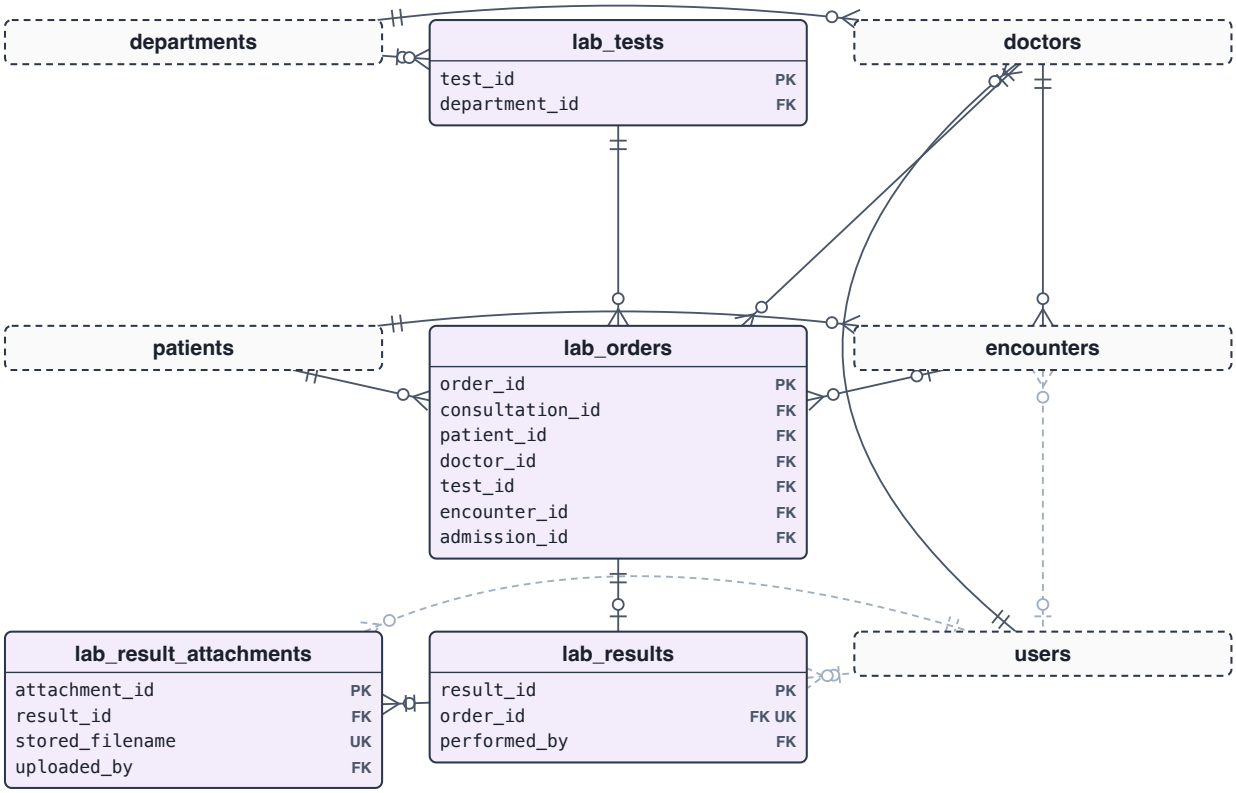
15 relationships among these tables; links to tables outside the figure are listed in section 3.

2.4 Module: Pharmacy



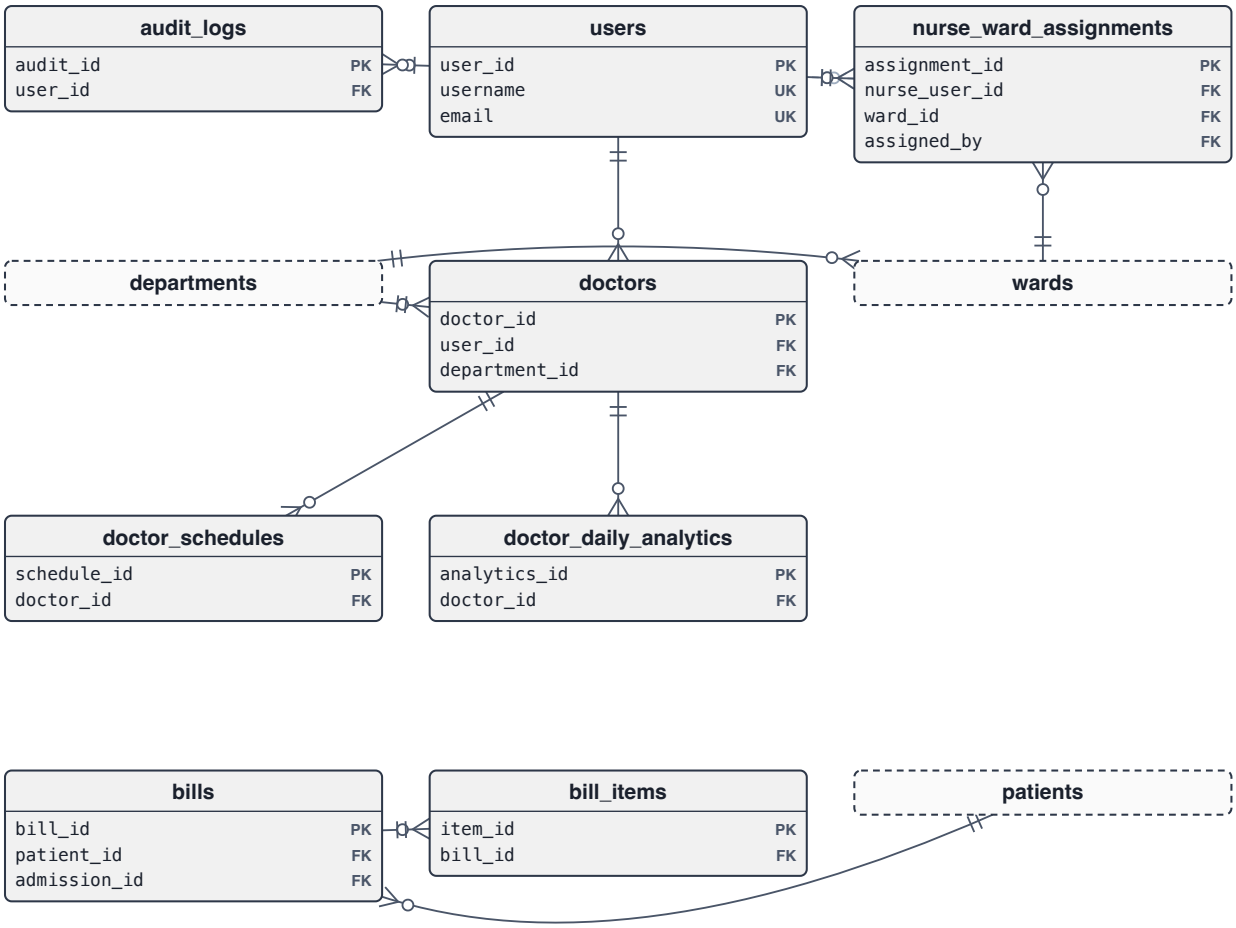
13 relationships among these tables; links to tables outside the figure are listed in section 3.

2.5 Module: Laboratory



14 relationships among these tables; links to tables outside the figure are listed in section 3.

2.6 Module: Staff, billing and audit



11 relationships among these tables; links to tables outside the figure are listed in section 3.

3. Relationships

Every foreign key, with the relationship it implements. *Cardinality* is parent : child:

- the parent side is 1, or 0..1 when the foreign key column allows NULL (the child may have no parent);
- the child side is N, or 0..1 when the foreign key column is unique (a 1:1 relationship).

Every ON UPDATE rule is NO ACTION, which InnoDB enforces as RESTRICT; primary keys are never updated. There are 74 foreign keys, 2 of them one-to-one.

Parent (1)	Child	Card.	Foreign key (constraint)	ON DELETE	ON UPD.
admissions	bed_assignment_log	1 : N	bed_assignment_log.admission_id → admissions.admission_id bed_assignment_log_ibfk_3	CASCADE	NO ACTION
	bills	0..1 : N	bills.admission_id → admissions.admission_id bills_ibfk_2	CASCADE	NO ACTION
	consultations	0..1 : N	consultations.admission_id → admissions.admission_id consultations_ibfk_4	CASCADE	NO ACTION
	encounters	0..1 : N	encounters.admission_id → admissions.admission_id fk_encounters_admission	RESTRICT	NO ACTION
	lab_orders	0..1 : N	lab_orders.admission_id → admissions.admission_id fk_lab_orders_admission	RESTRICT	NO ACTION
	medication_administrations	0..1 : N	medication_administrations.admission_id → admissions.admission_id fk_mar_admission	RESTRICT	NO ACTION
	prescriptions	0..1 : N	prescriptions.admission_id → admissions.admission_id fk_prescriptions_admission	RESTRICT	NO ACTION
	vital_signs	0..1 : N	vital_signs.admission_id → admissions.admission_id fk_vitals_admission	RESTRICT	NO ACTION
ambulances	emergency_cases	0..1 : N	emergency_cases.ambulance_id → ambulances.ambulance_id emergency_cases_ibfk_2	SET NULL	NO ACTION
appointments	consultations	0..1 : N	consultations.appointment_id → appointments.appointment_id consultations_ibfk_1	SET NULL	NO ACTION
	encounters	0..1 : 0..1	encounters.appointment_id → appointments.appointment_id fk_encounters_appointment	RESTRICT	NO ACTION
beds	admissions	1 : N	admissions.bed_id → beds.bed_id admissions_ibfk_3	RESTRICT	NO ACTION
	bed_assignment_log	1 : N	bed_assignment_log.bed_id → beds.bed_id bed_assignment_log_ibfk_1	RESTRICT	NO ACTION
	emergency_cases	0..1 : N	emergency_cases.assigned_bed_id → beds.bed_id emergency_cases_ibfk_4	SET NULL	NO ACTION
bills	bill_items	1 : N	bill_items.bill_id → bills.bill_id bill_items_ibfk_1	CASCADE	NO ACTION

Parent (1)	Child	Card.	Foreign key (constraint)	ON DELETE	ON UPD.
consultations	lab_orders	0..1 : N	lab_orders.consultation_id → consultations.consultation_id lab_orders_ibfk_1	SET NULL	NO ACTION
	prescriptions	0..1 : N	prescriptions.consultation_id → consultations.consultation_id prescriptions_ibfk_1	CASCADE	NO ACTION
departments	admissions	1 : N	admissions.department_id → departments.department_id admissions_ibfk_4	RESTRICT	NO ACTION
	appointments	1 : N	appointments.department_id → departments.department_id appointments_ibfk_3	RESTRICT	NO ACTION
	doctors	1 : N	doctors.department_id → departments.department_id doctors_ibfk_2	RESTRICT	NO ACTION
	lab_tests	0..1 : N	lab_tests.department_id → departments.department_id lab_tests_ibfk_1	SET NULL	NO ACTION
	wards	1 : N	wards.department_id → departments.department_id wards_ibfk_1	RESTRICT	NO ACTION
doctors	admissions	1 : N	admissions.doctor_id → doctors.doctor_id admissions_ibfk_2	RESTRICT	NO ACTION
	appointments	1 : N	appointments.doctor_id → doctors.doctor_id appointments_ibfk_2	RESTRICT	NO ACTION
	consultations	1 : N	consultations.doctor_id → doctors.doctor_id consultations_ibfk_3	RESTRICT	NO ACTION
	doctor_daily_analytics	1 : N	doctor_daily_analytics.doctor_id → doctors.doctor_id doctor_daily_analytics_ibfk_1	CASCADE	NO ACTION
	doctor_schedules	1 : N	doctor_schedules.doctor_id → doctors.doctor_id doctor_schedules_ibfk_1	CASCADE	NO ACTION
	emergency_cases	0..1 : N	emergency_cases.assigned_doctor_id → doctors.doctor_id emergency_cases_ibfk_3	SET NULL	NO ACTION
	encounters	1 : N	encounters.doctor_id → doctors.doctor_id fk_encounters_doctor	RESTRICT	NO ACTION
	lab_orders	1 : N	lab_orders.doctor_id → doctors.doctor_id lab_orders_ibfk_3	RESTRICT	NO ACTION
	prescriptions	1 : N	prescriptions.doctor_id → doctors.doctor_id prescriptions_ibfk_3	RESTRICT	NO ACTION
	surgery_requests	1 : N	surgery_requests.doctor_id → doctors.doctor_id surgery_requests_ibfk_2	CASCADE	NO ACTION
emergency_cases	admissions	0..1 : N	admissions.emergency_id → emergency_cases.emergency_id admissions_ibfk_5	SET NULL	NO ACTION

Parent (1)	Child	Card.	Foreign key (constraint)	ON DELETE	ON UPD.
encounters	consultations	0..1 : N	consultations.encounter_id → encounters.encounter_id fk_consultations_encounter	RESTRICT	NO ACTION
	lab_orders	0..1 : N	lab_orders.encounter_id → encounters.encounter_id fk_lab_orders_encounter	RESTRICT	NO ACTION
	prescriptions	0..1 : N	prescriptions.encounter_id → encounters.encounter_id fk_prescriptions_encounter	RESTRICT	NO ACTION
	surgery_requests	0..1 : N	surgery_requests.encounter_id → encounters.encounter_id fk_surgery_requests_encounter	RESTRICT	NO ACTION
	vital_signs	0..1 : N	vital_signs.encounter_id → encounters.encounter_id fk_vitals_encounter	RESTRICT	NO ACTION
lab_orders	lab_results	1 : 0..1	lab_results.order_id → lab_orders.order_id lab_results_ibfk_1	CASCADE	NO ACTION
lab_results	lab_result_attachments	1 : N	lab_result_attachments.result_id → lab_results.result_id lab_result_attachments_ibfk_1	CASCADE	NO ACTION
lab_tests	lab_orders	1 : N	lab_orders.test_id → lab_tests.test_id lab_orders_ibfk_4	RESTRICT	NO ACTION
medicine_batches	pharmacy_stock_movements	0..1 : N	pharmacy_stock_movements.batch_id → medicine_batches.batch_id fk_stock_movement_batch	RESTRICT	NO ACTION
	prescription_item_batches	1 : N	prescription_item_batches.batch_id → medicine_batches.batch_id fk_pib_batch	RESTRICT	NO ACTION
medicines	medicine_batches	1 : N	medicine_batches.medicine_id → medicines.medicine_id fk_medicine_batches_medicine	RESTRICT	NO ACTION
	pharmacy_stock_movements	1 : N	pharmacy_stock_movements.medicine_id → medicines.medicine_id pharmacy_stock_movements_ibfk_1	RESTRICT	NO ACTION
	prescription_items	1 : N	prescription_items.medicine_id → medicines.medicine_id prescription_items_ibfk_2	RESTRICT	NO ACTION
patients	admissions	1 : N	admissions.patient_id → patients.patient_id admissions_ibfk_1	RESTRICT	NO ACTION
	appointments	1 : N	appointments.patient_id → patients.patient_id appointments_ibfk_1	RESTRICT	NO ACTION
	bed_assignment_log	1 : N	bed_assignment_log.patient_id → patients.patient_id bed_assignment_log_ibfk_2	RESTRICT	NO ACTION
	bills	1 : N	bills.patient_id → patients.patient_id bills_ibfk_1	RESTRICT	NO ACTION
	consultations	1 : N	consultations.patient_id → patients.patient_id consultations_ibfk_2	RESTRICT	NO ACTION

Parent (1)	Child	Card.	Foreign key (constraint)	ON DELETE	ON UPD.
	emergency_cases	0..1 : N	emergency_cases.patient_id → patients.patient_id emergency_cases_ibfk_1	RESTRICT	NO ACTION
	encounters	1 : N	encounters.patient_id → patients.patient_id fk_encounters_patient	RESTRICT	NO ACTION
	lab_orders	1 : N	lab_orders.patient_id → patients.patient_id lab_orders_ibfk_2	RESTRICT	NO ACTION
	medication_administrations	1 : N	medication_administrations.patient_id → patients.patient_id fk_mar_patient	RESTRICT	NO ACTION
	prescriptions	1 : N	prescriptions.patient_id → patients.patient_id prescriptions_ibfk_2	RESTRICT	NO ACTION
	surgery_requests	1 : N	surgery_requests.patient_id → patients.patient_id surgery_requests_ibfk_1	CASCADE	NO ACTION
	vital_signs	1 : N	vital_signs.patient_id → patients.patient_id fk_vitals_patient	RESTRICT	NO ACTION
prescription_items	medication_administrations	1 : N	medication_administrations.prescription_item_id → prescription_items.item_id fk_mar_item	RESTRICT	NO ACTION
	prescription_item_batches	1 : N	prescription_item_batches.item_id → prescription_items.item_id fk_pib_item	RESTRICT	NO ACTION
prescriptions	prescription_items	1 : N	prescription_items.prescription_id → prescriptions.prescription_id prescription_items_ibfk_1	CASCADE	NO ACTION
users	audit_logs	0..1 : N	audit_logs.user_id → users.user_id fk_audit_logs_user	RESTRICT	NO ACTION
	doctors	1 : N	doctors.user_id → users.user_id doctors_ibfk_1	RESTRICT	NO ACTION
	encounters	0..1 : N	encounters.created_by → users.user_id fk_encounters_created_by	RESTRICT	NO ACTION
	lab_result_attachments	1 : N	lab_result_attachments.uploaded_by → users.user_id lab_result_attachments_ibfk_2	RESTRICT	NO ACTION
	lab_results	0..1 : N	lab_results.performed_by → users.user_id fk_lab_results_performed_by	SET NULL	NO ACTION
	medication_administrations	0..1 : N	medication_administrations.recorded_by → users.user_id fk_mar_recorded_by	RESTRICT	NO ACTION
	medicine_batches	0..1 : N	medicine_batches.received_by → users.user_id fk_medicine_batches_received_by	RESTRICT	NO ACTION
	nurse_ward_assignments	0..1 : N	nurse_ward_assignments.assigned_by → users.user_id fk_nwa_assigned_by	RESTRICT	NO ACTION

Parent (1)	Child	Card.	Foreign key (constraint)	ON DELETE	ON UPD.
	nurse_ward_assignments	1 : N	nurse_ward_assignments.nurse_user_id → users.user_id fk_nwa_nurse	RESTRICT	NO ACTION
	pharmacy_stock_movements	1 : N	pharmacy_stock_movements.performed_by → users.user_id pharmacy_stock_movements_ibfk_2	RESTRICT	NO ACTION
	vital_signs	1 : N	vital_signs.recorded_by → users.user_id fk_vitals_recorded_by	RESTRICT	NO ACTION
wards	beds	1 : N	beds.ward_id → wards.ward_id beds_ibfk_1	RESTRICT	NO ACTION
	nurse_ward_assignments	1 : N	nurse_ward_assignments.ward_id → wards.ward_id fk_nwa_ward	RESTRICT	NO ACTION

Many-to-many relationships and junction tables

Junction table	Joins (M:N)	Notes
prescription_items	prescriptions ⟷ medicines	A prescription lists many medicines; a medicine appears on many prescriptions. Carries the dose, frequency, duration, quantity and dispensed flag of each line.
prescription_item_batches	prescription_items ⟷ medicine_batches	A line can be filled from several batches, and a batch fills many lines. Pure junction with a quantity; unique (item_id, batch_id).
nurse_ward_assignments	users (nurses) ⟷ wards	Over time: a nurse covers several wards and a ward has several nurses, each assignment with start and end dates.
lab_orders	patients ⟷ lab_tests (by a doctor)	An associative entity: an order is a test for a patient by a doctor, with its own lifecycle and a 1:0..1 result.
bed_assignment_log	beds ⟷ admissions (over time)	History of which admission occupied which bed, written by triggers.
appointments	patients ⟷ doctors	An associative entity: each booking relates one patient and one doctor at a time slot.

One-to-one relationships (1 : 0..1):

- `lab_orders` → `lab_results`, through unique `lab_results.order_id`;
- `appointments` → `encounters`, through unique `encounters.appointment_id`.

`users` → `doctors` is intended to be 1:1, but `doctors.user_id` has no unique index, so the schema allows 1:N.

4. Table schemas

For each table: its purpose, its columns (type, NULL, default, key), then its keys, constraints and indexes, each index with the reason it exists.

- **Key column:** PK primary key, UK unique, FK foreign key, IDX first column of another index.
- **ENUM values** are listed under the type.
- **Defaults:** "—" means no default (a value is required); NULL means the column defaults to NULL.

Tables are grouped by module.

4.1 `patients`

Patients and visits · 36 rows in hospital_db

One row per registered patient: identity, demographics, contact and allergy information, and the optional ABHA (Ayushman Bharat Health Account) number. The MRN shown on screen is not stored; it is the formatted patient_id (MRN-000123).

Column	Type	Null	Default	Key	Extra
`patient_id`	`int`	no	—	PK	auto_increment
`name`	`varchar(100)`	no	—	IDX	
`date_of_birth`	`date`	no	—		
`dob_estimated`	`tinyint(1)`	no	`0`		
`gender`	`enum` `MALE · FEMALE · OTHER`	no	—		
`blood_group`	`enum` `A+ · A− · B+ · B− · AB+ · AB− · O+ · O−`	yes	NULL		
`phone`	`varchar(20)`	yes	NULL	IDX	
`address`	`text`	yes	NULL		
`emergency_contact`	`varchar(100)`	yes	NULL		
`created_at`	`datetime`	yes	CURRENT_TIMESTAMP		
`updated_at`	`datetime`	yes	CURRENT_TIMESTAMP		ON UPDATE now()
`allergies`	`text`	yes	NULL		
`abha_number`	`char(14)`	yes	NULL	UK	

Primary key	`patient_id`
Unique keys	`uq_patients_abha` (abha_number)
Foreign keys	none
CHECK constraints	`chk_patients_abha_format`: `((abha_number is null) or regexp_like(abha_number,'^[0-9]{14}$'))`

Index	Columns	Why it exists
`PRIMARY`	`patient_id`	Primary key: row identity. InnoDB stores the rows in this order (clustered index).
`idx_patients_name`	`name`	Name search and the duplicate check at registration. It helps "name = ?" and "name LIKE 'abc%'"; the list search "LIKE '%abc%'" cannot use it.
`idx_patients_phone`	`phone`	Exact phone match in the registration duplicate check.
`uq_patients_abha` UNIQUE	`abha_number`	One patient per ABHA number; NULLs are allowed many times, because most patients have none. A duplicate gives ER_DUP_ENTRY, answered 409.

4.2 appointments

Patients and visits · 9 rows in hospital_db

A booked outpatient slot: patient, doctor, date and time. The visit itself (arrival, triage, start, finish) lives in encounters; status here mirrors the visit.

Column	Type	Null	Default	Key	Extra
appointment_id	int	no	–	PK	auto_increment
patient_id	int	no	–	FK	
doctor_id	int	no	–	FK	
department_id	int	no	–	FK	
appointment_date	date	no	–		
appointment_time	time	no	–		
status	enum BOOKED · CHECKED_IN · IN_PROGRESS · COMPLETED · CANCELLED · NO_SHOW	yes	BOOKED		
reason	text	yes	NULL		
notes	text	yes	NULL		
created_at	datetime	yes	CURRENT_TIMESTAMP		
updated_at	datetime	yes	CURRENT_TIMESTAMP		ON UPDATE now()

Primary key	appointment_id
Unique keys	doctor_id (doctor_id, appointment_date, appointment_time)
Foreign keys	patient_id → patients(patient_id) ON DELETE RESTRICT doctor_id → doctors(doctor_id) ON DELETE RESTRICT department_id → departments(department_id) ON DELETE RESTRICT
CHECK constraints	none

Index	Columns	Why it exists
PRIMARY	appointment_id	Primary key: row identity. InnoDB stores the rows in this order (clustered index).
department_id	department_id	Supports the foreign key department_id → departments. InnoDB requires an index on the referencing column; it also serves joins from departments and the RESTRICT check when a row in departments is deleted.
doctor_id UNIQUE	doctor_id, appointment_date, appointment_time	UNIQUE (doctor_id, appointment_date, appointment_time): a doctor cannot be booked twice for the same slot. This is the database-level double-booking guard.
idx_appointments_doc_date_time	doctor_id, appointment_date, appointment_time	Same three columns as the unique key above, so it is redundant. Every insert and update maintains both, and nothing uses it that the unique key does not already serve. It is a candidate for removal (left alone here: no schema changes).
patient_id	patient_id	Supports the foreign key patient_id → patients. InnoDB requires an index on the referencing column; it also serves joins from patients and the RESTRICT check when a row in patients is deleted.

4.3 encounters

Patients and visits · 9 rows in hospital_db

A visit (FHIR Encounter): outpatient, inpatient or emergency, with its lifecycle PLANNED → ARRIVED → TRIAGED → IN_PROGRESS → FINISHED (or CANCELLED / ENTERED_IN_ERROR) and the time of each step. Notes, orders, prescriptions and vitals are filed under it.

Column	Type	Null	Default	Key	Extra
encounter_id	int	no	—	PK	auto_increment
patient_id	int	no	—	FK	
doctor_id	int	no	—	FK	
appointment_id	int	yes	NULL	UK FK	
admission_id	int	yes	NULL	FK	
encounter_type	enum OUTPATIENT · INPATIENT · EMERGENCY	no	OUTPATIENT		
status	enum PLANNED · ARRIVED · TRIAGED · IN_PROGRESS · FINISHED · CANCELLED · ENTERED_IN_ERROR	no	—		
arrived_at	datetime	yes	NULL		
triaged_at	datetime	yes	NULL		
start_timestamp	datetime	yes	NULL		
end_timestamp	datetime	yes	NULL		
created_by	int	yes	NULL	FK	
backfilled_from	varchar(40)	yes	NULL	UK	
created_at	datetime	no	CURRENT_TIMESTAMP		
updated_at	datetime	no	CURRENT_TIMESTAMP		ON UPDATE now()

Primary key	encounter_id
Unique keys	uq_encounters_appointment (appointment_id); uq_encounters_backfilled_from (backfilled_from)
Foreign keys	admission_id → admissions(admission_id) ON DELETE RESTRICT appointment_id → appointments(appointment_id) ON DELETE RESTRICT created_by → users(user_id) ON DELETE RESTRICT doctor_id → doctors(doctor_id) ON DELETE RESTRICT patient_id → patients(patient_id) ON DELETE RESTRICT
CHECK constraints	chk_encounters_times: ((end_timestamp is null) or (start_timestamp is null) or (end_timestamp >= start_timestamp))

Index	Columns	Why it exists
PRIMARY	encounter_id	Primary key: row identity. InnoDB stores the rows in this order (clustered index).
fk_encounters_created_by	created_by	Foreign key to users (who created the visit).
idx_encounters_admission	admission_id	Foreign key to admissions; the visits of an admission.
idx_encounters_doctor_status	doctor_id, status	A doctor's queue and "is my visit in progress?" (filed-under resolution for orders).
idx_encounters_patient_status	patient_id, status	A patient's open visit (nurse triage, the patient header, starting a visit).
uq_encounters_appointment UNIQUE	appointment_id	At most one visit per appointment (a 1:0..1 relationship). Checking the same appointment in twice fails here.
uq_encounters_backfilled_from UNIQUE	backfilled_from	Makes the data migrations idempotent: a re-run cannot create a second backfilled visit for the same source row.

4.4 consultations

Patients and visits · 3 rows in hospital_db

A doctor's clinical note (symptoms, diagnosis, assessment, plan), filed under a visit.

Column	Type	Null	Default	Key	Extra
consultation_id	int	no	—	PK	auto_increment
appointment_id	int	yes	NULL	FK	
patient_id	int	no	—	FK	
doctor_id	int	no	—	FK	
admission_id	int	yes	NULL	FK	
symptoms	text	no	—		
diagnosis	text	no	—		
assessment	text	yes	NULL		
plan	text	yes	NULL		
notes	text	yes	NULL		
consultation_time	datetime	no	—		
created_at	datetime	yes	CURRENT_TIMESTAMP		
encounter_id	int	yes	NULL	FK	

Primary key	consultation_id
Unique keys	none
Foreign keys	appointment_id → appointments(appointment_id) ON DELETE SET NULL patient_id → patients(patient_id) ON DELETE RESTRICT doctor_id → doctors(doctor_id) ON DELETE RESTRICT admission_id → admissions(admission_id) ON DELETE CASCADE encounter_id → encounters(encounter_id) ON DELETE RESTRICT
CHECK constraints	none

Index	Columns	Why it exists
PRIMARY	consultation_id	Primary key: row identity. InnoDB stores the rows in this order (clustered index).
admission_id	admission_id	Supports the foreign key admission_id → admissions. InnoDB requires an index on the referencing column; it also serves joins from admissions and the CASCADE check when a row in admissions is deleted.
appointment_id	appointment_id	Supports the foreign key appointment_id → appointments. InnoDB requires an index on the referencing column; it also serves joins from appointments and the SET NULL check when a row in appointments is deleted.
doctor_id	doctor_id	Supports the foreign key doctor_id → doctors. InnoDB requires an index on the referencing column; it also serves joins from doctors and the RESTRICT check when a row in doctors is deleted.
idx_consultations_encounter	encounter_id	The notes of a visit (Visits tab); foreign key to encounters.
patient_id	patient_id	Supports the foreign key patient_id → patients. InnoDB requires an index on the referencing column; it also serves joins from patients and the RESTRICT check when a row in patients is deleted.

4.5 `vital_signs`

Patients and visits · 2 rows in hospital_db

One vital-signs reading (temperature, pulse, respiratory rate, blood pressure, SpO2, pain score) filed under a visit, an admission or both. CHECK constraints reject impossible values; abnormal but possible values are stored and flagged by the app.

Column	Type	Null	Default	Key	Extra
vital_id	int	no	—	PK	auto_increment
patient_id	int	no	—	FK	
encounter_id	int	yes	NULL	FK	
admission_id	int	yes	NULL	FK	
recorded_at_utc	datetime	no	—		
recorded_by	int	no	—	FK	
temperature_c	decimal(4,1)	yes	NULL		
pulse_bpm	smallint	yes	NULL		
resp_rate	smallint	yes	NULL		
bp_systolic	smallint	yes	NULL		
bp_diastolic	smallint	yes	NULL		
spo2_pct	smallint	yes	NULL		
pain_score	tinyint	yes	NULL		
notes	varchar(255)	yes	NULL		
created_at	datetime	no	CURRENT_TIMESTAMP		

Primary key	vital_id
Unique keys	none
Foreign keys	admission_id → admissions(admission_id) ON DELETE RESTRICT encounter_id → encounters(encounter_id) ON DELETE RESTRICT patient_id → patients(patient_id) ON DELETE RESTRICT recorded_by → users(user_id) ON DELETE RESTRICT
CHECK constraints	chk_vitals_any: ((temperature_c is not null) or (pulse_bpm is not null) or (resp_rate is not null) or (bp_systolic is not null) or (spo2_pct is not null) or (pain_score is not null)) chk_vitals_bp_order: ((bp_systolic is null) or (bp_systolic > bp_diastolic)) chk_vitals_bp_pair: ((bp_systolic is null) = (bp_diastolic is null)) chk_vitals_context: ((encounter_id is not null) or (admission_id is not null)) chk_vitals_diastolic: ((bp_diastolic is null) or (bp_diastolic between 20 and 200)) chk_vitals_pain: ((pain_score is null) or (pain_score between 0 and 10)) chk_vitals_pulse: ((pulse_bpm is null) or (pulse_bpm between 20 and 250)) chk_vitals_resp_rate: ((resp_rate is null) or (resp_rate between 4 and 80)) chk_vitals_spo2: ((spo2_pct is null) or (spo2_pct between 50 and 100)) chk_vitals_systolic: ((bp_systolic is null) or (bp_systolic between 40 and 300)) chk_vitals_temperature: ((temperature_c is null) or (temperature_c between 25.0 and 45.0))

Index	Columns	Why it exists
PRIMARY	vital_id	Primary key: row identity. InnoDB stores the rows in this order (clustered index).
fk_vitals_recorded_by	recorded_by	Foreign key to users (the nurse who recorded it).
idx_vitals_admission	admission_id	Readings of an admission.
idx_vitals_encounter	encounter_id	Readings of a visit (Visits tab, last-vitals time in the nurse queue).

Index	Columns	Why it exists
idx_vitals_patient_time	patient_id, recorded_at_utc	The flowsheet: a patient's readings in time order (also covers the patient_id foreign key).

4.6 surgery_requests

Patients and visits · 0 rows in hospital_db

A request for a surgical procedure, with priority and status.

Column	Type	Null	Default	Key	Extra
request_id	int	no	—	PK	auto_increment
patient_id	int	no	—	FK	
doctor_id	int	no	—	FK	
procedure_name	varchar(255)	no	—		
diagnosis	text	no	—		
priority	enum ROUTINE · URGENT · EMERGENCY	yes	ROUTINE		
requested_date	date	no	—		
status	enum REQUESTED · SCHEDULED · IN_PROGRESS · COMPLETED · CANCELLED	yes	REQUESTED		
notes	text	yes	NULL		
created_at	datetime	yes	CURRENT_TIMESTAMP		
updated_at	datetime	yes	CURRENT_TIMESTAMP		ON UPDATE now()
encounter_id	int	yes	NULL	FK	

Primary key	request_id
Unique keys	none
Foreign keys	encounter_id → encounters(encounter_id) ON DELETE RESTRICT patient_id → patients(patient_id) ON DELETE CASCADE doctor_id → doctors(doctor_id) ON DELETE CASCADE
CHECK constraints	none

Index	Columns	Why it exists
PRIMARY	request_id	Primary key: row identity. InnoDB stores the rows in this order (clustered index).
doctor_id	doctor_id	Supports the foreign key doctor_id → doctors. InnoDB requires an index on the referencing column; it also serves joins from doctors and the CASCADE check when a row in doctors is deleted.
idx_surgery_requests_encounter	encounter_id	Foreign key to encounters; the requests of a visit.
patient_id	patient_id	Supports the foreign key patient_id → patients. InnoDB requires an index on the referencing column; it also serves joins from patients and the CASCADE check when a row in patients is deleted.

4.7 departments

Beds, admissions and emergency · 9 rows in hospital_db

Hospital departments (Cardiology, ICU, Emergency, …), referenced by doctors, wards, appointments, admissions and lab tests.

Column	Type	Null	Default	Key	Extra
department_id	int	no	–	PK	auto_increment
name	varchar(100)	no	–	UK	
description	text	yes	NULL		
floor	int	no	–		
phone	varchar(20)	yes	NULL		
is_active	tinyint(1)	yes	1		

Primary key	department_id
Unique keys	name (name)
Foreign keys	none
CHECK constraints	none

Index	Columns	Why it exists
PRIMARY	department_id	Primary key: row identity. InnoDB stores the rows in this order (clustered index).
name UNIQUE	name	UNIQUE: department names are unique.

4.8 wards

Beds, admissions and emergency · 6 rows in hospital_db

A ward belongs to a department and holds beds. Nurses are assigned to wards.

Column	Type	Null	Default	Key	Extra
ward_id	int	no	–	PK	auto_increment
name	varchar(100)	no	–		
department_id	int	no	–	FK	
floor	int	no	–		
capacity	int	no	–		
ward_type	varchar(50)	yes	NULL		

Primary key	ward_id
Unique keys	none
Foreign keys	department_id → departments(department_id) ON DELETE RESTRICT
CHECK constraints	none

Index	Columns	Why it exists
PRIMARY	ward_id	Primary key: row identity. InnoDB stores the rows in this order (clustered index).
department_id	department_id	Supports the foreign key department_id → departments. InnoDB requires an index on the referencing column; it also serves joins from departments and the RESTRICT check when a row in departments is deleted.

4.9 beds

Beds, admissions and emergency · 35 rows in hospital_db

A physical bed: ward, code (e.g. ICU-04), type and status. status is OCCUPIED while the bed has an active admission (kept by triggers).

Column	Type	Null	Default	Key	Extra
bed_id	int	no	—	PK	auto_increment
ward_id	int	no	—	FK	
bed_number	varchar(20)	no	—		
floor	int	no	—		
bed_type	enum GENERAL · ICU · NICU · ISOLATION · VENTILATOR_ICU	no	—		
status	enum AVAILABLE · OCCUPIED · MAINTENANCE · RESERVED	yes	AVAILABLE	IDX	
has_ventilator	tinyint(1)	yes	0		

Primary key	bed_id
Unique keys	ward_id (ward_id, bed_number)
Foreign keys	ward_id → wards(ward_id) ON DELETE RESTRICT
CHECK constraints	none

Index	Columns	Why it exists
PRIMARY	bed_id	Primary key: row identity. InnoDB stores the rows in this order (clustered index).
idx_beds_status_type	status, bed_type	Finding an available bed of a type (emergency allocation, admission dialog, available_beds_view).
idx_beds_ward_status	ward_id, status	The beds of a ward with their status (nurse station, ward summaries). Also covers the ward_id foreign key.
ward_id UNIQUE	ward_id, bed_number	UNIQUE (ward_id, bed_number): a bed code is unique within its ward.

4.10 admissions

Beds, admissions and emergency · 19 rows in hospital_db

An inpatient stay: patient, admitting doctor, bed, department, optional emergency case, dates and status (ACTIVE / DISCHARGED / TRANSFERRED).

Column	Type	Null	Default	Key	Extra
admission_id	int	no	—	PK	auto_increment
patient_id	int	no	—	FK	
doctor_id	int	no	—	FK	
bed_id	int	no	—	FK	
department_id	int	no	—	FK	
emergency_id	int	yes	NULL	FK	
admission_date	datetime	no	—		
discharge_date	datetime	yes	NULL		
status	enum ACTIVE · DISCHARGED · TRANSFERRED	yes	ACTIVE		
diagnosis	text	no	—		
notes	text	yes	NULL		
created_at	datetime	yes	CURRENT_TIMESTAMP		
updated_at	datetime	yes	CURRENT_TIMESTAMP		ON UPDATE now()

Primary key	`admission_id`
Unique keys	none
Foreign keys	`patient_id` → `patients(patient_id)` ON DELETE RESTRICT `doctor_id` → `doctors(doctor_id)` ON DELETE RESTRICT `bed_id` → `beds(bed_id)` ON DELETE RESTRICT `department_id` → `departments(department_id)` ON DELETE RESTRICT `emergency_id` → `emergency_cases(emergency_id)` ON DELETE SET NULL
CHECK constraints	`admissions_chk_1`: `((discharge_date is null) or (discharge_date >= admission_date))`

Index	Columns	Why it exists
`PRIMARY`	`admission_id`	Primary key: row identity. InnoDB stores the rows in this order (clustered index).
`department_id`	`department_id`	Supports the foreign key department_id → departments. InnoDB requires an index on the referencing column; it also serves joins from departments and the RESTRICT check when a row in departments is deleted.
`emergency_id`	`emergency_id`	Supports the foreign key emergency_id → emergency_cases. InnoDB requires an index on the referencing column; it also serves joins from emergency_cases and the SET NULL check when a row in emergency_cases is deleted.
`idx_admissions_bed_status`	`bed_id, status`	The active admission in a bed (nurse station join: a.bed_id = b.bed_id AND a.status = 'ACTIVE'). Also covers the bed_id foreign key.
`idx_admissions_doctor_status`	`doctor_id, status`	A doctor's active admissions (dashboard; deciding which admission an order is filed under). Also covers the doctor_id foreign key.
`idx_admissions_patient_status`	`patient_id, status`	"Is this patient admitted?" and the patient's active admission. Also covers the patient_id foreign key.

4.11 `bed_assignment_log`

Beds, admissions and emergency · 19 rows in hospital_db

History of bed occupancy: one row per admission, opened by the after_admission_insert trigger and closed on discharge.

Column	Type	Null	Default	Key	Extra
`log_id`	`int`	no	–	PK	auto_increment
`bed_id`	`int`	no	–	FK	
`patient_id`	`int`	no	–	FK	
`admission_id`	`int`	no	–	FK	
`assigned_at`	`datetime`	no	–		
`released_at`	`datetime`	yes	NULL		
`status`	`enum` `ACTIVE · RELEASED`	yes	ACTIVE		

Primary key	`log_id`
Unique keys	none
Foreign keys	`bed_id` → `beds(bed_id)` ON DELETE RESTRICT `patient_id` → `patients(patient_id)` ON DELETE RESTRICT `admission_id` → `admissions(admission_id)` ON DELETE CASCADE
CHECK constraints	none

Index	Columns	Why it exists
PRIMARY	log_id	Primary key: row identity. InnoDB stores the rows in this order (clustered index).
admission_id	admission_id	Supports the foreign key admission_id → admissions. InnoDB requires an index on the referencing column; it also serves joins from admissions and the CASCADE check when a row in admissions is deleted.
bed_id	bed_id	Supports the foreign key bed_id → beds. InnoDB requires an index on the referencing column; it also serves joins from beds and the RESTRICT check when a row in beds is deleted.
patient_id	patient_id	Supports the foreign key patient_id → patients. InnoDB requires an index on the referencing column; it also serves joins from patients and the RESTRICT check when a row in patients is deleted.

4.12 emergency_cases

Beds, admissions and emergency · 9 rows in hospital_db

An emergency arrival: severity, symptoms, requirements (specialization, bed type, ventilator), status, and the doctor and bed it was allocated. patient_id is NULL for an arrival not yet registered.

Column	Type	Null	Default	Key	Extra
emergency_id	int	no	—	PK	auto_increment
patient_id	int	yes	NULL	FK	
ambulance_id	int	yes	NULL	FK	
severity	enum CRITICAL · VERY_SERIOUS · SERIOUS · MODERATE · STABLE	no	—		
symptoms	text	no	—		
arrival_time	datetime	no	—		
required_specialization	varchar(100)	yes	NULL		
required_bed_type	enum GENERAL · ICU · NICU · ISOLATION · VENTILATOR_ICU	yes	NULL		
ventilator_required	tinyint(1)	yes	0		
status	enum WAITING · TRIAGED · ALLOCATED · ADMITTED · DISCHARGED · CANCELLED	yes	WAITING	IDX	
assigned_doctor_id	int	yes	NULL	FK	
assigned_bed_id	int	yes	NULL	FK	
notes	text	yes	NULL		
created_at	datetime	yes	CURRENT_TIMESTAMP		
updated_at	datetime	yes	CURRENT_TIMESTAMP		ON UPDATE now()

Primary key	emergency_id
Unique keys	none
Foreign keys	patient_id → patients(patient_id) ON DELETE RESTRICT ambulance_id → ambulances(ambulance_id) ON DELETE SET NULL assigned_doctor_id → doctors(doctor_id) ON DELETE SET NULL assigned_bed_id → beds(bed_id) ON DELETE SET NULL

CHECK constraints	none

Index	Columns	Why it exists
`PRIMARY`	`emergency_id`	Primary key: row identity. InnoDB stores the rows in this order (clustered index).
`ambulance_id`	`ambulance_id`	Supports the foreign key ambulance_id → ambulances. InnoDB requires an index on the referencing column; it also serves joins from ambulances and the SET NULL check when a row in ambulances is deleted.
`assigned_bed_id`	`assigned_bed_id`	Supports the foreign key assigned_bed_id → beds. InnoDB requires an index on the referencing column; it also serves joins from beds and the SET NULL check when a row in beds is deleted.
`assigned_doctor_id`	`assigned_doctor_id`	Supports the foreign key assigned_doctor_id → doctors. InnoDB requires an index on the referencing column; it also serves joins from doctors and the SET NULL check when a row in doctors is deleted.
`idx_emergency_status_severity_time`	`status, severity, arrival_time`	The emergency queue: open statuses, ordered by severity and then arrival time.
`patient_id`	`patient_id`	Supports the foreign key patient_id → patients. InnoDB requires an index on the referencing column; it also serves joins from patients and the RESTRICT check when a row in patients is deleted.

4.13 `ambulances`

Beds, admissions and emergency · 6 rows in hospital_db

Ambulance fleet: vehicle, driver and status.

Column	Type	Null	Default	Key	Extra
`ambulance_id`	`int`	no	—	PK	auto_increment
`vehicle_number`	`varchar(20)`	no	—	UK	
`driver_name`	`varchar(100)`	no	—		
`driver_phone`	`varchar(20)`	no	—		
`current_location`	`varchar(255)`	yes	NULL		
`status`	`enum` `AVAILABLE · EN_ROUTE · ARRIVED · IN_SERVICE · MAINTENANCE`	yes	AVAILABLE		

Primary key	`ambulance_id`
Unique keys	`vehicle_number` (vehicle_number)
Foreign keys	none
CHECK constraints	none

Index	Columns	Why it exists
`PRIMARY`	`ambulance_id`	Primary key: row identity. InnoDB stores the rows in this order (clustered index).
`vehicle_number` UNIQUE	`vehicle_number`	UNIQUE: one row per registration number.

4.14 `medicines`

Pharmacy · 22 rows in hospital_db

The medicine catalogue with the current stock total. stock_quantity is the sum of the medicine's batches (kept equal in every stock transaction).

Column	Type	Null	Default	Key	Extra
medicine_id	int	no	—	PK	auto_increment
name	varchar(150)	no	—		
category	varchar(100)	no	—		
manufacturer	varchar(150)	yes	NULL		
stock_quantity	int	yes	0		
reorder_level	int	no	50		
unit_price	decimal(10,2)	no	—		
expiry_date	date	no	—		
requires_prescription	tinyint(1)	yes	1		
created_at	datetime	yes	CURRENT_TIMESTAMP		
updated_at	datetime	yes	CURRENT_TIMESTAMP		ON UPDATE now()

Primary key	medicine_id
Unique keys	none
Foreign keys	none
CHECK constraints	medicines_chk_1: (stock_quantity >= 0) medicines_chk_2: (unit_price >= 0)

Index	Columns	Why it exists
PRIMARY	medicine_id	Primary key: row identity. InnoDB stores the rows in this order (clustered index).

4.15 medicine_batches

Pharmacy · 22 rows in hospital_db

A lot of a medicine with its own expiry date and quantity. Dispensing takes stock first-expired-first-out (FEFO).

Column	Type	Null	Default	Key	Extra
batch_id	int	no	—	PK	auto_increment
medicine_id	int	no	—	FK	
batch_number	varchar(50)	no	—		
quantity	int	no	—		
expiry_date	date	no	—		
received_date	datetime	no	CURRENT_TIMESTAMP		
received_by	int	yes	NULL	FK	
created_at	datetime	no	CURRENT_TIMESTAMP		
updated_at	datetime	no	CURRENT_TIMESTAMP		ON UPDATE now()

Primary key	batch_id
Unique keys	uq_medicine_batches_number (medicine_id, batch_number)
Foreign keys	medicine_id → medicines(medicine_id) ON DELETE RESTRICT received_by → users(user_id) ON DELETE RESTRICT
CHECK constraints	chk_medicine_batches_quantity: (quantity >= 0)

Index	Columns	Why it exists
PRIMARY	batch_id	Primary key: row identity. InnoDB stores the rows in this order (clustered index).
fk_medicine_batches_received_by	received_by	Foreign key to users (who received it).
idx_medicine_batches_fefo	medicine_id, expiry_date, batch_id	(medicine_id, expiry_date, batch_id): serves "the batches of this medicine, earliest expiry first" with FOR UPDATE. Lock order follows this index, so batch locks are always taken in the same order (no deadlock).
uq_medicine_batches_number UNIQUE	medicine_id, batch_number	A batch number is unique per medicine.

4.16 prescriptions

Pharmacy · 4 rows in hospital_db

A prescription header: patient, prescribing doctor, date, status, and the visit or admission it was written under.

Column	Type	Null	Default	Key	Extra
prescription_id	int	no	—	PK	auto_increment
consultation_id	int	yes	NULL	FK	
patient_id	int	no	—	FK	
doctor_id	int	no	—	FK	
prescription_date	datetime	no	—		
status	enum CREATED · DISPENSED · PARTIALLY_DISPENSED · CANCELLED	yes	CREATED		
notes	text	yes	NULL		
created_at	datetime	yes	CURRENT_TIMESTAMP		
encounter_id	int	yes	NULL	FK	
admission_id	int	yes	NULL	FK	

Primary key	prescription_id
Unique keys	none
Foreign keys	admission_id → admissions(admission_id) ON DELETE RESTRICT encounter_id → encounters(encounter_id) ON DELETE RESTRICT consultation_id → consultations(consultation_id) ON DELETE CASCADE patient_id → patients(patient_id) ON DELETE RESTRICT doctor_id → doctors(doctor_id) ON DELETE RESTRICT
CHECK constraints	none

Index	Columns	Why it exists
PRIMARY	prescription_id	Primary key: row identity. InnoDB stores the rows in this order (clustered index).
consultation_id	consultation_id	Supports the foreign key consultation_id → consultations. InnoDB requires an index on the referencing column; it also serves joins from consultations and the CASCADE check when a row in consultations is deleted.
doctor_id	doctor_id	Supports the foreign key doctor_id → doctors. InnoDB requires an index on the referencing column; it also serves joins from doctors and the RESTRICT check when a row in doctors is deleted.
idx_prescriptions_admission	admission_id	Prescriptions filed under an admission.
idx_prescriptions_encounter	encounter_id	Prescriptions of a visit.

Index	Columns	Why it exists
`idx_prescriptions_patient_status`	`patient_id, status`	A patient's prescriptions by status; covers the patient_id foreign key.

4.17 `prescription_items`

Pharmacy · 13 rows in hospital_db

One medicine line on a prescription: the dose as written ("250 mg"), frequency, duration, quantity, whether it was dispensed, and optional structured fields (frequency_code, duration_days, route, units_per_dose) used to schedule ward doses.

Column	Type	Null	Default	Key	Extra
`item_id`	`int`	no	—	PK	auto_increment
`prescription_id`	`int`	no	—	FK	
`medicine_id`	`int`	no	—	FK	
`dosage`	`varchar(50)`	no	—		
`frequency`	`varchar(50)`	no	—		
`duration`	`varchar(50)`	no	—		
`quantity`	`int`	no	—		
`dispensed`	`tinyint(1)`	yes	`0`		
`dispensed_at`	`datetime`	yes	NULL		
`frequency_code`	`enum` `OD · BD · TDS · QID · Q6H · Q8H · STAT · PRN`	yes	NULL		
`duration_days`	`smallint`	yes	NULL		
`route`	`enum` `ORAL · IV · IM · SC · SUBLINGUAL · INHALED · TOPICAL · RECTAL · OTHER`	yes	NULL		
`units_per_dose`	`smallint`	yes	NULL		

Primary key	`item_id`
Unique keys	none
Foreign keys	`prescription_id` → `prescriptions(prescription_id)` ON DELETE CASCADE `medicine_id` → `medicines(medicine_id)` ON DELETE RESTRICT
CHECK constraints	`chk_prescription_items_structured`: `(((duration_days is null) or (duration_days > 0)) and ((units_per_dose is null) or (units_per_dose > 0)))` `prescription_items_chk_1`: `(quantity > 0)`

Index	Columns	Why it exists
`PRIMARY`	`item_id`	Primary key: row identity. InnoDB stores the rows in this order (clustered index).
`medicine_id`	`medicine_id`	Supports the foreign key medicine_id → medicines. InnoDB requires an index on the referencing column; it also serves joins from medicines and the RESTRICT check when a row in medicines is deleted.
`prescription_id`	`prescription_id`	Supports the foreign key prescription_id → prescriptions. InnoDB requires an index on the referencing column; it also serves joins from prescriptions and the CASCADE check when a row in prescriptions is deleted.

4.18 prescription_item_batches

Pharmacy · 4 rows in hospital_db

Which batches each dispensed line came from, and how much from each (traceability for recalls).

Column	Type	Null	Default	Key	Extra
id	int	no	—	PK	auto_increment
item_id	int	no	—	FK	
batch_id	int	no	—	FK	
quantity	int	no	—		
created_at	datetime	no	CURRENT_TIMESTAMP		

Primary key	id
Unique keys	uq_prescription_item_batches (item_id, batch_id)
Foreign keys	batch_id → medicine_batches(batch_id) ON DELETE RESTRICT item_id → prescription_items(item_id) ON DELETE RESTRICT
CHECK constraints	chk_pib_quantity: (quantity > 0)

Index	Columns	Why it exists
PRIMARY	id	Primary key: row identity. InnoDB stores the rows in this order (clustered index).
idx_prescription_item_batches_batch	batch_id	Foreign key to medicine_batches; "who received this batch?" (recall).
uq_prescription_item_batches UNIQUE	item_id, batch_id	One row per (item, batch) pair.

4.19 pharmacy_stock_movements

Pharmacy · 6 rows in hospital_db

The stock ledger: every receipt, adjustment and dispense, with signed quantity, batch, reason, reference (e.g. the prescription id) and who did it.

Column	Type	Null	Default	Key	Extra
movement_id	int	no	—	PK	auto_increment
medicine_id	int	no	—	FK	
movement_type	enum RECEIPT · ADJUSTMENT · DISPENSE	no	—		
quantity	int	no	—		
reason	varchar(50)	yes	NULL		
notes	text	yes	NULL		
performed_by	int	no	—	FK	
created_at	datetime	no	CURRENT_TIMESTAMP		
reference_id	int	yes	NULL		
batch_id	int	yes	NULL	FK	

Primary key	movement_id
Unique keys	none
Foreign keys	batch_id → medicine_batches(batch_id) ON DELETE RESTRICT medicine_id → medicines(medicine_id) ON DELETE RESTRICT performed_by → users(user_id) ON DELETE RESTRICT

CHECK constraints	none

Index	Columns	Why it exists
`PRIMARY`	`movement_id`	Primary key: row identity. InnoDB stores the rows in this order (clustered index).
`idx_stock_movement_batch`	`batch_id`	The ledger of a batch; covers the batch_id foreign key.
`idx_stock_movement_medicine`	`medicine_id`	The ledger of a medicine; covers the medicine_id foreign key.
`performed_by`	`performed_by`	Supports the foreign key performed_by → users. InnoDB requires an index on the referencing column; it also serves joins from users and the RESTRICT check when a row in users is deleted.

4.20 `medication_administrations`

Pharmacy · 15 rows in hospital_db

The bedside Medication Administration Record: each scheduled dose of a dispensed order for an admitted patient, and what happened (ADMINISTERED, REFUSED, MISSED, CANCELLED) with who and when. Times are UTC.

Column	Type	Null	Default	Key	Extra
`administration_id`	`int`	no	—	PK	auto_increment
`prescription_item_id`	`int`	no	—	FK	
`patient_id`	`int`	no	—	FK	
`admission_id`	`int`	yes	NULL	FK	
`scheduled_at_utc`	`datetime`	yes	NULL		
`status`	`enum` `PENDING · ADMINISTERED · REFUSED · MISSED · CANCELLED`	no	PENDING		
`administered_at_utc`	`datetime`	yes	NULL		
`dose_given`	`varchar(50)`	yes	NULL		
`route_given`	`enum` `ORAL · IV · IM · SC · SUBLINGUAL · INHALED · TOPICAL · RECTAL · OTHER`	yes	NULL		
`reason`	`varchar(255)`	yes	NULL		
`recorded_by`	`int`	yes	NULL	FK	
`recorded_at_utc`	`datetime`	yes	NULL		
`created_at`	`datetime`	no	CURRENT_TIMESTAMP		
`updated_at`	`datetime`	no	CURRENT_TIMESTAMP		ON UPDATE now()

Primary key	`administration_id`
Unique keys	`uq_mar_item_slot` (prescription_item_id, scheduled_at_utc)
Foreign keys	`admission_id` → `admissions(admission_id)` ON DELETE RESTRICT `prescription_item_id` → `prescription_items(item_id)` ON DELETE RESTRICT `patient_id` → `patients(patient_id)` ON DELETE RESTRICT `recorded_by` → `users(user_id)` ON DELETE RESTRICT
CHECK constraints	`chk_mar_given`: `((status <> 'ADMINISTERED') or (administered_at_utc is not null))`

Index	Columns	Why it exists
`PRIMARY`	`administration_id`	Primary key: row identity. InnoDB stores the rows in this order (clustered index).
`fk_mar_recorded_by`	`recorded_by`	Foreign key to users (the nurse who recorded it).

Index	Columns	Why it exists
`idx_mar_admission_status`	`admission_id, status`	Pending doses of an admission (cancelled at discharge).
`idx_mar_patient_time`	`patient_id, scheduled_at_utc`	A patient's doses in time order (the grid).
`uq_mar_item_slot` UNIQUE	`prescription_item_id, scheduled_at_utc`	One dose per order per scheduled time: scheduling twice (e.g. a retried dispense) cannot create duplicate doses.

4.21 `lab_tests`

Laboratory · 15 rows in hospital_db

The test catalogue: name, category, department, cost, unit, the reference range as text, and numeric reference and critical bounds used to flag results automatically.

Column	Type	Null	Default	Key	Extra
`test_id`	`int`	no	—	PK	auto_increment
`name`	`varchar(150)`	no	—		
`category`	`varchar(100)`	no	—		
`department_id`	`int`	yes	NULL	FK	
`normal_range`	`varchar(100)`	yes	NULL		
`unit`	`varchar(50)`	yes	NULL		
`cost`	`decimal(10,2)`	no	—		
`reference_low`	`decimal(14,4)`	yes	NULL		
`reference_high`	`decimal(14,4)`	yes	NULL		
`critical_low`	`decimal(14,4)`	yes	NULL		
`critical_high`	`decimal(14,4)`	yes	NULL		

Primary key	`test_id`
Unique keys	none
Foreign keys	`department_id` → `departments(department_id)` ON DELETE SET NULL
CHECK constraints	`lab_tests_chk_1`: `(cost >= 0)`

Index	Columns	Why it exists
`PRIMARY`	`test_id`	Primary key: row identity. InnoDB stores the rows in this order (clustered index).
`department_id`	`department_id`	Supports the foreign key department_id → departments. InnoDB requires an index on the referencing column; it also serves joins from departments and the SET NULL check when a row in departments is deleted.

4.22 `lab_orders`

Laboratory · 9 rows in hospital_db

One ordered test for a patient: test, ordering doctor, status (ORDERED → PROCESSING → COMPLETED), priority, and the visit or admission it was filed under.

Column	Type	Null	Default	Key	Extra
`order_id`	`int`	no	—	PK	auto_increment
`consultation_id`	`int`	yes	NULL	FK	
`patient_id`	`int`	no	—	FK	
`doctor_id`	`int`	no	—	FK	

Column	Type	Null	Default	Key	Extra
test_id	int	no	—	FK	
order_date	datetime	no	—		
status	enum ORDERED · SAMPLE_COLLECTED · PROCESSING · COMPLETED · CANCELLED	yes	ORDERED		
priority	enum NORMAL · URGENT · STAT	yes	NORMAL		
notes	text	yes	NULL		
created_at	datetime	yes	CURRENT_TIMESTAMP		
updated_at	datetime	yes	CURRENT_TIMESTAMP		ON UPDATE now()
encounter_id	int	yes	NULL	FK	
admission_id	int	yes	NULL	FK	

Primary key	order_id
Unique keys	none
Foreign keys	admission_id → admissions(admission_id) ON DELETE RESTRICT encounter_id → encounters(encounter_id) ON DELETE RESTRICT consultation_id → consultations(consultation_id) ON DELETE SET NULL patient_id → patients(patient_id) ON DELETE RESTRICT doctor_id → doctors(doctor_id) ON DELETE RESTRICT test_id → lab_tests(test_id) ON DELETE RESTRICT
CHECK constraints	none

Index	Columns	Why it exists
PRIMARY	order_id	Primary key: row identity. InnoDB stores the rows in this order (clustered index).
consultation_id	consultation_id	Supports the foreign key consultation_id → consultations. InnoDB requires an index on the referencing column; it also serves joins from consultations and the SET NULL check when a row in consultations is deleted.
doctor_id	doctor_id	Supports the foreign key doctor_id → doctors. InnoDB requires an index on the referencing column; it also serves joins from doctors and the RESTRICT check when a row in doctors is deleted.
idx_lab_orders_admission	admission_id	Orders filed under an admission.
idx_lab_orders_encounter	encounter_id	Orders of a visit.
idx_lab_orders_patient_status	patient_id, status	A patient's orders by status; covers the patient_id foreign key.
test_id	test_id	Supports the foreign key test_id → lab_tests. InnoDB requires an index on the referencing column; it also serves joins from lab_tests and the RESTRICT check when a row in lab_tests is deleted.

4.23 lab_results

Laboratory · 6 rows in hospital_db

The result of an order: the value (as text and, if numeric, as a number), unit, the reference bounds used, the interpretation (NORMAL / LOW / HIGH / CRITICAL) and whether it was computed or entered by hand.

Column	Type	Null	Default	Key	Extra
result_id	int	no	—	PK	auto_increment

Column	Type	Null	Default	Key	Extra
order_id	int	no	—	UK FK	
result_value	text	no	—		
unit	varchar(50)	yes	NULL		
reference_range	varchar(100)	yes	NULL		
interpretation	enum NORMAL · LOW · HIGH · CRITICAL	yes	NULL		
result_date	datetime	no	—		
technician_notes	text	yes	NULL		
performed_by	int	yes	NULL	FK	
reviewed_by_doctor	tinyint(1)	yes	0		
reviewed_at	datetime	yes	NULL		
created_at	datetime	yes	CURRENT_TIMESTAMP		
numeric_value	decimal(14,4)	yes	NULL		
reference_low	decimal(14,4)	yes	NULL		
reference_high	decimal(14,4)	yes	NULL		
interpretation_source	enum AUTO · MANUAL	yes	NULL		

Primary key	result_id
Unique keys	order_id (order_id)
Foreign keys	performed_by → users(user_id) ON DELETE SET NULL order_id → lab_orders(order_id) ON DELETE CASCADE
CHECK constraints	none

Index	Columns	Why it exists
PRIMARY	result_id	Primary key: row identity. InnoDB stores the rows in this order (clustered index).
fk_lab_results_performed_by	performed_by	Foreign key to users (the technician).
order_id UNIQUE	order_id	UNIQUE: at most one result per order (a 1:0..1 relationship).

4.24 lab_result_attachments

Laboratory · 1 rows in hospital_db

Uploaded report files (PDF) for a result. The file lives on disk outside any public folder; the row stores only a generated stored_filename and metadata.

Column	Type	Null	Default	Key	Extra
attachment_id	int	no	—	PK	auto_increment
result_id	int	no	—	FK	
original_filename	varchar(255)	no	—		
stored_filename	varchar(255)	no	—	UK	
mime_type	varchar(100)	no	—		
file_size	int	no	—		
uploaded_by	int	no	—	FK	
uploaded_at	datetime	no	CURRENT_TIMESTAMP		

Primary key	attachment_id

Unique keys	`stored_filename` (stored_filename)
Foreign keys	`result_id` → `lab_results(result_id)` ON DELETE CASCADE `uploaded_by` → `users(user_id)` ON DELETE RESTRICT
CHECK constraints	none

Index	Columns	Why it exists
`PRIMARY`	`attachment_id`	Primary key: row identity. InnoDB stores the rows in this order (clustered index).
`idx_lab_attachment_result`	`result_id`	The attachments of a result; covers the result_id foreign key.
`stored_filename` UNIQUE	`stored_filename`	UNIQUE: two rows can never point at the same file on disk.
`uploaded_by`	`uploaded_by`	Supports the foreign key uploaded_by → users. InnoDB requires an index on the referencing column; it also serves joins from users and the RESTRICT check when a row in users is deleted.

4.25 `users`

Staff, billing and audit · 17 rows in hospital_db

Staff accounts: username, bcrypt password hash, role (ADMIN, DOCTOR, RECEPTIONIST, NURSE, LABORATORY, PHARMACY), name and contact. Accounts are deactivated (is_active = 0), never deleted.

Column	Type	Null	Default	Key	Extra
`user_id`	`int`	no	–	PK	auto_increment
`username`	`varchar(50)`	no	–	UK	
`password_hash`	`varchar(255)`	no	–		
`role`	`enum` `ADMIN · DOCTOR · RECEPTIONIST · NURSE · LABORATORY · PHARMACY`	no	–		
`full_name`	`varchar(100)`	no	–		
`email`	`varchar(100)`	no	–	UK	
`phone`	`varchar(20)`	no	–		
`is_active`	`tinyint(1)`	yes	1		
`created_at`	`datetime`	yes	`CURRENT_TIMESTAMP`		
`updated_at`	`datetime`	yes	`CURRENT_TIMESTAMP`		ON UPDATE now()

Primary key	`user_id`
Unique keys	`email` (email); `username` (username)
Foreign keys	none
CHECK constraints	none

Index	Columns	Why it exists
`PRIMARY`	`user_id`	Primary key: row identity. InnoDB stores the rows in this order (clustered index).
`email` UNIQUE	`email`	UNIQUE: one account per email.
`username` UNIQUE	`username`	UNIQUE: login name.

4.26 `doctors`

Staff, billing and audit · 12 rows in hospital_db

The doctor profile of a DOCTOR user: department, specialization, status, shift and workload (current_workload is kept by the admission triggers).

Column	Type	Null	Default	Key	Extra
doctor_id	int	no	—	PK	auto_increment
user_id	int	no	—	FK	
department_id	int	no	—	FK	
name	varchar(100)	no	—		
specialization	varchar(100)	no	—	IDX	
phone	varchar(20)	yes	NULL		
status	enum AVAILABLE · BUSY · OFF_DUTY · ON_LEAVE	yes	AVAILABLE		
shift	enum MORNING · AFTERNOON · NIGHT	no	—		
current_workload	int	yes	0		
max_workload	int	yes	5		
created_at	datetime	yes	CURRENT_TIMESTAMP		
updated_at	datetime	yes	CURRENT_TIMESTAMP		ON UPDATE now()

Primary key	doctor_id
Unique keys	none
Foreign keys	user_id → users(user_id) ON DELETE RESTRICT department_id → departments(department_id) ON DELETE RESTRICT
CHECK constraints	doctors_chk_1: ((current_workload <= max_workload) and (current_workload >= 0))

Index	Columns	Why it exists
PRIMARY	doctor_id	Primary key: row identity. InnoDB stores the rows in this order (clustered index).
department_id	department_id	Supports the foreign key department_id → departments. InnoDB requires an index on the referencing column; it also serves joins from departments and the RESTRICT check when a row in departments is deleted.
idx_doctors_spec_status	specialization, status	Finding an available doctor of a specialization (emergency allocation, booking list).
user_id	user_id	Foreign key to users. Not unique, so the database would accept two doctor profiles for one account (a weakness: the intended 1:1 is enforced only by the account-creation code).

4.27 doctor_schedules

Staff, billing and audit · 36 rows in hospital_db

Weekly working hours of each doctor.

Column	Type	Null	Default	Key	Extra
schedule_id	int	no	—	PK	auto_increment
doctor_id	int	no	—	FK	
day_of_week	enum MONDAY · TUESDAY · WEDNESDAY · THURSDAY · FRIDAY · SATURDAY · SUNDAY	no	—		
start_time	time	no	—		
end_time	time	no	—		
is_available	tinyint(1)	yes	1		

Primary key	`schedule_id`
Unique keys	`doctor_id` (doctor_id, day_of_week, start_time)
Foreign keys	`doctor_id` → `doctors(doctor_id)` ON DELETE CASCADE
CHECK constraints	none

Index	Columns	Why it exists
`PRIMARY`	`schedule_id`	Primary key: row identity. InnoDB stores the rows in this order (clustered index).
`doctor_id` UNIQUE	`doctor_id, day_of_week, start_time`	UNIQUE (doctor_id, day_of_week, start_time): no two slots start at the same time on the same day. Also covers the doctor_id foreign key.

4.28 `doctor_daily_analytics`

Staff, billing and audit · 4 rows in hospital_db

Per-doctor daily counters (patients seen, cases solved, pending, …) shown on the doctor dashboard. Recomputed from the source tables on every dashboard load and upserted.

Column	Type	Null	Default	Key	Extra
`analytics_id`	`int`	no	—	PK	auto_increment
`doctor_id`	`int`	no	—	FK	
`analytics_date`	`date`	no	—		
`patients_seen`	`int`	yes	`0`		
`emergency_cases`	`int`	yes	`0`		
`appointment_patients`	`int`	yes	`0`		
`patients_treated`	`int`	yes	`0`		
`cases_solved`	`int`	yes	`0`		
`cases_postponed`	`int`	yes	`0`		
`cases_cancelled`	`int`	yes	`0`		
`cases_pending`	`int`	yes	`0`		
`created_at`	`datetime`	yes	`CURRENT_TIMESTAMP`		
`updated_at`	`datetime`	yes	`CURRENT_TIMESTAMP`		ON UPDATE now()

Primary key	`analytics_id`
Unique keys	`unique_doctor_date` (doctor_id, analytics_date)
Foreign keys	`doctor_id` → `doctors(doctor_id)` ON DELETE CASCADE
CHECK constraints	none

Index	Columns	Why it exists
`PRIMARY`	`analytics_id`	Primary key: row identity. InnoDB stores the rows in this order (clustered index).
`unique_doctor_date` UNIQUE	`doctor_id, analytics_date`	One row per doctor per day; the target of INSERT ... ON DUPLICATE KEY UPDATE. Also covers the doctor_id foreign key.

4.29 `nurse_ward_assignments`

Staff, billing and audit · 2 rows in hospital_db

Which nurse covers which ward from start_date to end_date (open-ended when NULL). A nurse's "assigned patients" are derived from it.

Column	Type	Null	Default	Key	Extra
assignment_id	int	no	—	PK	auto_increment
nurse_user_id	int	no	—	FK	
ward_id	int	no	—	FK	
start_date	date	no	—		
end_date	date	yes	NULL		
assigned_by	int	yes	NULL	FK	
created_at	datetime	no	CURRENT_TIMESTAMP		
updated_at	datetime	no	CURRENT_TIMESTAMP		ON UPDATE now()

Primary key	assignment_id
Unique keys	none
Foreign keys	assigned_by → users(user_id) ON DELETE RESTRICT nurse_user_id → users(user_id) ON DELETE RESTRICT ward_id → wards(ward_id) ON DELETE RESTRICT
CHECK constraints	chk_nwa_dates: ((end_date is null) or (end_date >= start_date))

Index	Columns	Why it exists
PRIMARY	assignment_id	Primary key: row identity. InnoDB stores the rows in this order (clustered index).
fk_nwa_assigned_by	assigned_by	Foreign key to users (the administrator who assigned it).
idx_nwa_nurse_dates	nurse_user_id, start_date, end_date	"Wards assigned to this nurse today" (start_date <= today and end_date NULL or >= today), used in every nurse access check. Also covers the nurse foreign key.
idx_nwa_ward	ward_id	Foreign key to wards.

4.30 bills

Staff, billing and audit · 4 rows in hospital_db

A bill for a patient, optionally for an admission, with total, amount paid and status.

Column	Type	Null	Default	Key	Extra
bill_id	int	no	—	PK	auto_increment
patient_id	int	no	—	FK	
admission_id	int	yes	NULL	FK	
bill_date	datetime	no	—		
total_amount	decimal(12,2)	yes	0.00		
paid_amount	decimal(12,2)	yes	0.00		
status	enum PENDING · PARTIAL · PAID · CANCELLED	yes	PENDING		
payment_method	varchar(50)	yes	NULL		
notes	text	yes	NULL		
created_at	datetime	yes	CURRENT_TIMESTAMP		
updated_at	datetime	yes	CURRENT_TIMESTAMP		ON UPDATE now()

Primary key	bill_id
Unique keys	none

Foreign keys	`patient_id` → `patients(patient_id)` ON DELETE RESTRICT `admission_id` → `admissions(admission_id)` ON DELETE CASCADE
CHECK constraints	`bills_chk_1`: `(total_amount >= 0)` `bills_chk_2`: `((paid_amount >= 0) and (paid_amount <= total_amount))`

Index	Columns	Why it exists
`PRIMARY`	`bill_id`	Primary key: row identity. InnoDB stores the rows in this order (clustered index).
`admission_id`	`admission_id`	Supports the foreign key admission_id → admissions. InnoDB requires an index on the referencing column; it also serves joins from admissions and the CASCADE check when a row in admissions is deleted.
`idx_bills_patient_status`	`patient_id, status`	A patient's bills by status (pending bills); covers the patient_id foreign key.

4.31 `bill_items`

Staff, billing and audit · 19 rows in hospital_db

Lines of a bill: description, category, quantity, unit and total price.

Column	Type	Null	Default	Key	Extra
`item_id`	`int`	no	—	PK	auto_increment
`bill_id`	`int`	no	—	FK	
`description`	`varchar(255)`	no	—		
`category`	`enum` `CONSULTATION · BED · ICU · LABORATORY · MEDICINE · PROCEDURE · AMBULANCE · OTHER`	no	—		
`quantity`	`int`	yes	1		
`unit_price`	`decimal(10,2)`	no	—		
`total_price`	`decimal(10,2)`	no	—		
`created_at`	`datetime`	yes	`CURRENT_TIMESTAMP`		

Primary key	`item_id`
Unique keys	none
Foreign keys	`bill_id` → `bills(bill_id)` ON DELETE CASCADE

CHECK constraints	`bill_items_chk_1`: `(quantity > 0)`
	`bill_items_chk_2`: `(unit_price >= 0)`
	`bill_items_chk_3`: `(total_price >= 0)`

Index	Columns	Why it exists
`PRIMARY`	`item_id`	Primary key: row identity. InnoDB stores the rows in this order (clustered index).
`bill_id`	`bill_id`	Supports the foreign key bill_id → bills. InnoDB requires an index on the referencing column; it also serves joins from bills and the CASCADE check when a row in bills is deleted.

4.32 `audit_logs`

Staff, billing and audit · 3552 rows in hospital_db

The append-only audit trail: who (user and role at the time), what action, on which entity and patient, the outcome (SUCCESS / DENIED), request details, and a JSON object of ids and changed field names. It never holds passwords, tokens or note text. Triggers reject UPDATE and DELETE.

Column	Type	Null	Default	Key	Extra
audit_id	bigint	no	—	PK	auto_increment
user_id	int	yes	NULL	FK	
role	varchar(20)	yes	NULL		
action	varchar(64)	no	—	IDX	
entity_type	varchar(50)	yes	NULL		
entity_id	int	yes	NULL		
patient_id	int	yes	NULL	IDX	
outcome	enum SUCCESS · DENIED	no	SUCCESS	IDX	
status_code	smallint	yes	NULL		
method	varchar(10)	yes	NULL		
path	varchar(255)	yes	NULL		
ip_address	varchar(45)	yes	NULL		
details	json	yes	NULL		
created_at	datetime(3)	no	CURRENT_TIMESTAMP(3)	IDX	

Primary key	audit_id
Unique keys	none
Foreign keys	user_id → users(user_id) ON DELETE RESTRICT
CHECK constraints	none

Index	Columns	Why it exists
PRIMARY	audit_id	Primary key: row identity. InnoDB stores the rows in this order (clustered index).
idx_audit_action_created	action, created_at	Filter by action over time.
idx_audit_created	created_at	The audit viewer's time-range listing.
idx_audit_outcome_created	outcome, created_at	Denied requests over time (the "denied access, last 24 hours" figure).
idx_audit_patient_created	patient_id, created_at	Filter by patient over time ("who looked at this record?").
idx_audit_user_created	user_id, created_at	Filter by user over time; covers the user_id foreign key.

4.33 schema_migrations

Staff, billing and audit · 15 rows in hospital_db

Which migration files have been applied, and when. Written only by server/scripts/migrate.js.

Column	Type	Null	Default	Key	Extra
version	varchar(255)	no	—	PK	
applied_at	datetime	no	CURRENT_TIMESTAMP		
note	varchar(100)	yes	NULL		

Primary key	version
Unique keys	none
Foreign keys	none
CHECK constraints	none

Index	Columns	Why it exists
`PRIMARY`	`version`	Primary key: row identity. InnoDB stores the rows in this order (clustered index).

5. Functional dependencies and candidate keys

An FD X → Y means that rows with equal X always have equal Y. Every table has a surrogate key that determines all its attributes. The other FDs listed below come from unique keys and from business rules, including the ones that cause the 3NF and BCNF exceptions in section 6. A unique column that allows NULL identifies a row only when it is not NULL, so it is noted but not counted as a candidate key.

Table	Functional dependencies	Candidate keys
`patients`	`patient_id → every other attribute` `abha_number → patient_id (when not NULL)`	patient_id. abha_number is unique but nullable, so it identifies a patient only when present; it is not a candidate key in the strict sense.

`appointments`	`appointment_id → every other attribute`
	`(doctor_id, appointment_date, appointment_time) → appointment_id`
	`doctor_id → department_id (only at booking time: the server copies the doctor's department)`
		appointment_id; (doctor_id, appointment_date, appointment_time).

`encounters`	`encounter_id → every other attribute`
	`appointment_id → encounter_id (when not NULL; unique)`
	`backfilled_from → encounter_id (when not NULL; unique)`
	`appointment_id → patient_id, doctor_id (the appointment's)`
		encounter_id. appointment_id and backfilled_from are unique but nullable.

`consultations`	`consultation_id → every other attribute`
	`encounter_id → patient_id, doctor_id`
	`appointment_id → patient_id, doctor_id`
		consultation_id.

`vital_signs`	`vital_id → every other attribute`
	`encounter_id → patient_id`
	`admission_id → patient_id`
		vital_id.

`surgery_requests`	`request_id → every other attribute` `encounter_id → patient_id, doctor_id`	request_id.
`departments`	`department_id → every other attribute` `name → department_id`	department_id; name.
`wards`	`ward_id → every other attribute`	ward_id. (name is not unique.)

`beds`	`bed_id → every other attribute`
	`(ward_id, bed_number) → bed_id`
	`admissions (active) → status: a derived value`
		bed_id; (ward_id, bed_number).

`admissions`	`admission_id → every other attribute` `emergency_id → patient_id (the emergency case's patient)`	admission_id.
`bed_assignment_log`	`log_id → every other attribute` `admission_id → bed_id, patient_id`	log_id. (admission_id is not declared unique, although the triggers write one row per admission.)
`emergency_cases`	`emergency_id → every other attribute`	emergency_id.
`ambulances`	`ambulance_id → every other attribute` `vehicle_number → ambulance_id`	ambulance_id; vehicle_number.
`medicines`	`medicine_id → every other attribute` `SUM(medicine_batches.quantity) → stock_quantity (derived across tables)`	medicine_id. (name is not unique.)
`medicine_batches`	`batch_id → every other attribute` `(medicine_id, batch_number) → batch_id`	batch_id; (medicine_id, batch_number).

Table	Functional dependencies	Candidate keys
prescriptions	prescription_id → every other attribute encounter_id → patient_id, doctor_id admission_id → patient_id items (all dispensed) → status = DISPENSED (derived)	prescription_id.
prescription_items	item_id → every other attribute	item_id.
prescription_item_batches	id → every other attribute (item_id, batch_id) → quantity	id; (item_id, batch_id).
pharmacy_stock_movements	movement_id → every other attribute batch_id → medicine_id	movement_id.
medication_administrations	administration_id → every other attribute (prescription_item_id, scheduled_at_utc) → administration_id prescription_item_id → patient_id	administration_id; (prescription_item_id, scheduled_at_utc) (scheduled_at_utc is NULL for unscheduled PRN doses).
lab_tests	test_id → every other attribute	test_id.
lab_orders	order_id → every other attribute encounter_id → patient_id, doctor_id admission_id → patient_id consultation_id → patient_id, doctor_id	order_id.
lab_results	result_id → every other attribute order_id → result_id (numeric_value, reference_low, reference_high, critical bounds) → interpretation (when interpretation_source = AUTO)	result_id; order_id.
lab_result_attachments	attachment_id → every other attribute stored_filename → attachment_id	attachment_id; stored_filename.
users	user_id → every other attribute username → user_id email → user_id	user_id; username; email.
doctors	doctor_id → every other attribute user_id → doctor_id (intended; not declared unique) active admissions → current_workload (derived)	doctor_id. user_id should be a second candidate key (1:1 with users), but there is no unique index on it.
doctor_schedules	schedule_id → every other attribute (doctor_id, day_of_week, start_time) → schedule_id	schedule_id; (doctor_id, day_of_week, start_time).
doctor_daily_analytics	analytics_id → every other attribute (doctor_id, analytics_date) → analytics_id appointments, emergency_cases, admissions of that day → every counter (derived)	analytics_id; (doctor_id, analytics_date).
nurse_ward_assignments	assignment_id → every other attribute	assignment_id. (No uniqueness on nurse, ward and dates: overlapping duplicate assignments are possible and harmless.)
bills	bill_id → every other attribute SUM(bill_items.total_price) → total_amount (derived) admission_id → patient_id	bill_id.
bill_items	item_id → every other attribute (quantity, unit_price) → total_price	item_id.
audit_logs	audit_id → every other attribute	audit_id.
schema_migrations	version → every other attribute	version.

6. Normalization

6.1 Method

Each table is judged strictly:

- **1NF:** atomic values, no repeating groups.
- **2NF:** no non-key attribute depends on part of a candidate key.
- **3NF:** no non-key attribute depends on another non-key attribute (no transitive dependency).
- **BCNF:** every determinant is a candidate key.

Verdicts:

- **Y** holds;
- **N** does not hold, and the explanation names the dependency;
- **Y*** holds within the table, but a stated business rule would break it if enforced, or the table stores a value derived from other tables.

Tables whose only key is a single surrogate column satisfy 2NF automatically for that key; 2NF matters here only for their composite candidate keys.

6.2 Verdict per table

Table	1NF	2NF	3NF	BCNF	Why
`patients`	N	Y	Y	Y	1NF is not strictly met: allergies is free text that may hold a list ("Penicillin, Sulfa"), a repeating group inside one column. The app treats it as one note (or the fixed value "No known allergies") and never queries inside it. Apart from that, every non-key attribute depends only on patient_id, so 2NF, 3NF and BCNF hold.
`appointments`	Y	Y*	Y*	Y*	department_id is copied from doctors when the appointment is booked. If "an appointment's department is always its doctor's department" were a rule, doctor_id → department_id would be a partial dependency on the composite candidate key (failing 2NF), and a transitive one from appointment_id (failing 3NF). It is kept on purpose as a snapshot: if a doctor later changes department, past appointments still show where they happened. status is a mirror of encounters.status (section 6.3).
`encounters`	Y	Y	N	N	patient_id and doctor_id are also determined by appointment_id when there is one (a transitive dependency). They are stored because walk-in and emergency visits have no appointment, and because every queue query filters on them. The server checks they match the appointment when it creates the visit, under the appointment row lock.
`consultations`	Y	Y	N	N	patient_id and doctor_id are determined by encounter_id (and by appointment_id), so they depend on the key transitively. They pre-date encounters (the original schema) and are kept so that old notes without a visit still have an owner. The server files new notes under the doctor's in-progress visit and checks that the patient and doctor match.
`vital_signs`	Y	Y	N	N	patient_id is determined by encounter_id or admission_id, whichever is set. It is stored because the flowsheet query reads a patient's readings over time (idx_vitals_patient_time) without joining two optional parents. The server resolves the visit or admission for the given patient, so the two cannot disagree.
`surgery_requests`	Y	Y	N	N	Same pattern as consultations: patient_id and doctor_id are also reachable through encounter_id (nullable, added later). This table has no rows in hospital_db.
`departments`	Y	Y	Y	Y	Every attribute depends on the whole key and nothing else; name is a second candidate key, which BCNF allows.

Table	1NF	2NF	3NF	BCNF	Why
`wards`	Y	Y	Y	Y	All attributes depend only on ward_id. capacity is the planned capacity, entered by hand, not a count of beds, so it is not derived. It can disagree with the actual number of beds, and nothing checks it.
`beds`	Y	Y	Y*	Y*	Within the table, 3NF and BCNF hold. Two caveats: floor may repeat the ward's floor (if a ward is always on one floor, ward_id → floor is a partial dependency on the composite candidate key), and status = OCCUPIED duplicates "there is an ACTIVE admission for this bed" (a cached value, section 6.3).
`admissions`	Y	Y	Y*	Y*	department_id is supplied when admitting. If it were always the department of the bed's ward, bed_id → ward → department_id would make it transitive; emergency allocation does set it that way, but the admission form lets the front desk choose. For emergency admissions, patient_id repeats emergency_cases.patient_id. Rule not enforced in the database: one ACTIVE admission per patient and per bed (no partial unique index in MySQL). It is enforced by the server under row locks, and checked by query 11.
`bed_assignment_log`	Y	Y	N	N	bed_id and patient_id are determined by admission_id (transfers are not modelled), so they are transitive. The table is written only by triggers, from the admission row in the same statement, so the copies cannot drift. Removing them would leave the history unreadable if an admission's bed were ever changed.
`emergency_cases`	Y	Y	Y	Y	All attributes describe the case. assigned_doctor_id and assigned_bed_id repeat what the resulting admission records, but they are the allocation decision and are set in the same transaction.
`ambulances`	Y	Y	Y	Y	Driver details are stored per vehicle (one current driver), so they depend on the key. If drivers were shared between vehicles, a drivers table would be needed.
`medicines`	Y	Y	Y*	Y*	Within the row everything depends on medicine_id. stock_quantity is a deliberately stored aggregate of medicine_batches (section 6.3). expiry_date is a leftover from before batches existed: each batch now has its own expiry, so this column is not maintained as "earliest expiry" and should not be trusted.
`medicine_batches`	Y	Y	Y	Y	quantity, expiry_date and received_* depend on the whole lot. No determinant other than a candidate key.
`prescriptions`	Y	Y	N	N	patient_id is determined by encounter_id or admission_id, and doctor_id by encounter_id, so both are transitive. They are kept because older prescriptions have no visit, and because pharmacy and patient queries filter on them directly. The server checks that the visit belongs to the same patient and doctor. status partly summarises the items; it is set in the same transaction as the items.
`prescription_items`	Y	Y	Y*	Y*	This table links prescriptions and medicines (M:N) and carries attributes of the link. frequency and frequency_code, and duration and duration_days, are two representations of the same fact (free text kept as written, structured for scheduling). They can disagree, and nothing in the database ties them together.
`prescription_item_batches`	Y	Y	Y	Y	A junction table between prescription_items and medicine_batches; quantity depends on the pair.
`pharmacy_stock_movements`	Y	Y	N	N	medicine_id is determined by batch_id when there is one (rows from before batches have batch_id NULL). It is kept so that the ledger of a medicine is one indexed query. reference_id is polymorphic: what it points to depends on movement_type, so it cannot be a foreign key.
`medication_administrations`	Y	N	N	N	patient_id depends on prescription_item_id alone. That is a partial dependency on the composite candidate key (fails 2NF), and a transitive one from administration_id (item → prescription → patient). It is copied when the dose is created, because the nurse grid and the access check read doses by patient. admission_id is the admission at scheduling time (a snapshot).
`lab_tests`	Y	Y	Y*	Y*	normal_range (text) and reference_low/reference_high (numbers) state the same fact twice. The numbers were backfilled from the text where it was unambiguous, and they can drift apart.

Table	1NF	2NF	3NF	BCNF	Why
`lab_orders`	Y	Y	N	N	Same as prescriptions: patient_id and doctor_id are reachable through encounter_id, admission_id or consultation_id, which makes them transitive. They are kept for direct filtering and for orders written before visits existed. The order is an associative entity between patients, doctors and lab_tests.
`lab_results`	Y	Y	Y	N	For automatic rows, interpretation is a function of the stored value and bounds: a determinant that is not a key, so BCNF does not hold. 3NF holds in this case, because interpretation is not reached through another key. reference_low/high are copied from lab_tests on purpose: a result must keep the range it was judged against even if the catalogue changes later. reference_range (text) repeats the same range for display.
`lab_result_attachments`	Y	Y	Y	Y	All attributes describe one stored file.
`users`	Y	Y	Y	Y	Three candidate keys, each a determinant, which is allowed in BCNF.
`doctors`	Y	Y	Y*	Y*	name repeats users.full_name for the linked account; this is cross-table redundancy, and nothing keeps the two in sync. current_workload is a cached count of the doctor's active admissions, maintained by triggers (section 6.3).
`doctor_schedules`	Y	Y	Y	Y	end_time and is_available depend on the whole slot. Overlapping slots (09:00–13:00 and 10:00–12:00) are not prevented.
`doctor_daily_analytics`	Y	Y	Y*	Y*	Structurally in BCNF, but every counter is derived from other tables: it is a cache (section 6.3), not authoritative data.
`nurse_ward_assignments`	Y	Y	Y	Y	A temporal junction between users (nurses) and wards, with its own attributes.
`bills`	Y	Y	N	N	total_amount is the sum of the bill's items, and patient_id is determined by admission_id when set. Both are stored copies. The billing code adds each item's amount to total_amount in the same transaction as the item. A bill can also be created with a starting total, so total_amount is not guaranteed to equal the sum of its items, and nothing in the database checks it. The CHECK paid_amount <= total_amount relies on the stored total.
`bill_items`	Y	Y	N	N	total_price is computed from quantity and unit_price, two non-key attributes, so it is a transitive dependency. A generated column (total_price AS (quantity * unit_price)) would remove the redundancy; it is stored. The app always writes quantity 1 with total_price = unit_price; sp_generate_bill writes quantity 1 as well.
`audit_logs`	N	Y	Y	Y	details is a JSON document. Taken as a single opaque value it is atomic, but it holds structured content, so strictly it is not 1NF. It is never joined on. role is a snapshot of users.role at the time: deliberate, because an audit row must not change when the account later changes. patient_id has no foreign key on purpose: the trail must outlive any change to patient records.
`schema_migrations`	Y	Y	Y	Y	A key and two facts about it.

Summary: 13 of 33 tables are in BCNF without qualification. 11 are not in 3NF, all of them because they store a copy of a value reachable through another reference, or a value computed from other columns. 1 table is not in 2NF, and 2 are not strictly in 1NF (a free-text list and a JSON document).

6.3 Deliberate denormalization

Most tables are in BCNF, and most of the exceptions follow one of four deliberate patterns. Each stores something that could be derived, to make an important read cheap or to keep a historical value. Each has a named anomaly risk and a named mechanism that prevents it.

Stored value	Anomaly it risks	What keeps it consistent
`medicines.stock_quantity`, the total, stored alongside `medicine_batches.quantity`	Update anomaly: a stock change that updates one but not the other leaves the total disagreeing with the batches, so the pharmacy sees stock that does not exist (or misses stock that does).	• Every stock change (receipt, adjustment, dispense) updates both in one transaction, with the medicine row locked (FOR UPDATE, ascending medicine_id) before its batches. • Dispensing marks the items dispensed, and the `after_prescription_dispense` trigger takes the same quantity off `stock_quantity`; the service takes it off the batches in the same transaction. • `CHECK (stock_quantity >= 0)` and the `before_medicine_stock_update` trigger reject a negative total. • The invariant is checked by query 10 (GROUP BY … HAVING) after every stock test, and by the stress test (300 rounds × 2 scenarios): 0 mismatches.
`appointments.status` mirroring `encounters.status`	Update anomaly: the appointment list says "Checked in" while the visit is already "In progress" (or the other way round).	• The visit is the source of truth. • Every transition goes through one function, `transitionEncounter`, which locks the appointment row first, then the encounter row (lock order appointment → encounter), and writes both statuses in the same transaction from a fixed mirror table: ARRIVED→CHECKED_IN, IN_PROGRESS→IN_PROGRESS, FINISHED→COMPLETED, CANCELLED/ENTERED_IN_ERROR→CANCELLED. • The old `PUT /appointments/:id/status` route uses the same function. • Concurrency tests race two transitions and check the mirror holds.
`doctor_daily_analytics` (daily counters), `doctors.current_workload`, `beds.status`	Stale reads: counts that no longer match the appointments and admissions they summarise. For `current_workload` or `beds.status` this would let the allocator over-assign a doctor or double-book a bed.	• `doctor_daily_analytics` is a cache: it is recomputed from the source tables each time a doctor opens the dashboard, and upserted on `UNIQUE (doctor_id, analytics_date)`. Nothing makes decisions from it. • `current_workload` and `beds.status` are maintained by the admission triggers in the same statement as the admission insert or discharge. • `CHECK (current_workload BETWEEN 0 AND max_workload)` stops it drifting out of range. • Migrations 004 and 005 repaired beds left OCCUPIED by seed data that bypassed the trigger path; the queries there show how drift is found.
Copied `patient_id` (and `doctor_id`) on rows that also reference a visit, admission or order	A row could name one patient while its visit belongs to another.	• The server resolves the visit or admission *for the given patient and doctor* (`resolveEncounterForRecord`, `resolveOrderContext`) under a locking read, and refuses a mismatch. • Copies taken at creation are never updated afterwards (no code path changes a row's patient).

Copies kept for history are not anomalies, because they must *not* follow later changes: `lab_results.reference_low/high` (the range the result was judged against), `appointments.department_id`, `audit_logs.role` and `medication_administrations.admission_id`.

6.4 Worked example: from an unnormalized record to these tables

Start from one paper record, as a clerk might keep it for an outpatient visit (unnormalized form, UNF):

Patient	Phone	Doctor	Department	Visit	Symptoms / diagnosis	Medicines (dose, freq, qty)	Tests (result)
Aarav Sharma, M, 1985-02-02	98765 43210	Dr. John Smith	Cardiology, floor 2	2026-10-11 09:30	Chest tightness / suspected angina	Aspirin 75 mg, OD, 30; Nitroglycerin 0.4 mg, PRN, 10	Lipid profile (pending); Blood glucose fasting (128 mg/dL, HIGH)

UNF → 1NF. The medicines and tests columns are repeating groups, and "patient" packs three facts into one cell. Split every cell into one atomic value and every repeated item into its own row. The record becomes one row per (visit, medicine, test) combination, with the key (patient, visit time, medicine, test):

name	dob	sex	phone	doctor	dept	floor	time	diag.	medicine	dose	freq	qty	test	cost	result
Aarav Sharma	1985-02-02	M	98765 43210	Dr. John Smith	Cardio-logy	2	09:30	angina?	Aspirin	75 mg	OD	30	Lipid profile	600	—
… four rows in all (2 medicines × 2 tests): every patient and visit fact is repeated four times.															

Anomalies at this stage:

- **Update:** changing the phone means editing four rows.
- **Insertion:** a test cannot be recorded without a medicine.
- **Deletion:** removing the last medicine loses the diagnosis.

1NF → 2NF (remove dependencies on part of the key):

- patient facts depend only on the patient;
- visit facts depend only on (patient, visit time);
- dose, frequency and quantity depend on (visit, medicine);
- the result depends on (visit, test);
- cost depends on the test alone.

So the relation splits into PATIENT(patient, dob, gender, phone), VISIT(visit, patient, doctor, dept, dept_floor, time, symptoms, diagnosis), VISIT_MEDICINE(visit, medicine, dose, freq, qty), VISIT_TEST(visit, test, result) and TEST(test, cost). A surrogate key is added for each entity (patient_id, encounter_id, …), because names are not unique.

2NF → 3NF (remove transitive dependencies):

- In VISIT, dept and dept_floor depend on the doctor, not on the visit (visit → doctor → department → floor). That gives DOCTOR(doctor_id, name, department_id) and DEPARTMENT(department_id, name, floor).
- A visit's note is written by the doctor, so symptoms and diagnosis move to a note (consultation) of the visit. One visit can then have several notes.
- Medicines become a catalogue, MEDICINE(medicine_id, name, …), and what a doctor prescribes is grouped under a prescription header.

BCNF. Every remaining determinant is a key:

- (prescription, medicine) lines get their own id;
- a lab result is determined by its order;
- a test's cost by the test.

The result is the tables this system actually has:

Normalized relation	Table in hospital_db
PATIENT	`patients` (patient_id, name, date_of_birth, gender, phone, …)
DEPARTMENT, DOCTOR	`departments`; `doctors` (department_id) linked to `users`
VISIT	`appointments` (the booking) and `encounters` (the visit and its timeline)
Note of a visit	`consultations` (encounter_id, symptoms, diagnosis, assessment, plan)
MEDICINE; VISIT_MEDICINE	`medicines`; `prescriptions` (encounter_id) + `prescription_items` (medicine_id, dosage, frequency, quantity)
TEST; VISIT_TEST	`lab_tests` (cost); `lab_orders` (encounter_id, test_id) + `lab_results` (order_id, value)

The worked example also shows where the live schema steps back from 3NF on purpose. `lab_orders` and `prescriptions` keep `patient_id` next to `encounter_id`, which is the transitive copy the 3NF step removed. Section 6.3 explains why, and what keeps the copy correct.

7. Views

There are nine views. Eight come from the original project; `v_current_stock` was added with medicine batches. All are `SQL SECURITY DEFINER`, with definer `root@localhost` (MySQL records this). The app account already has SELECT on the database, so the definer gives it nothing extra. *Updatable* below is MySQL's own verdict (`information_schema.VIEWS.IS_UPDATABLE`):

- views with GROUP BY or aggregates are never updatable;
- join views marked YES can be updated through one base table at a time, but not inserted into;
- `emergency_queue_view` (two LEFT JOINs) is reported NO. MySQL does not record why, and outer joins are the likely reason.

Only `v_current_stock` is used by the application.

7.1 `available_beds_view`

Purpose	Beds with status AVAILABLE, with ward and department names. Read-only in practice; not used by the API (the admission dialog queries beds directly).
Tables	`beds`, `wards`, `departments`
Updatable	YES (UPDATE through one base table at a time; no INSERT through a join)

```
SELECT b.bed_id AS bed_id,
  b.bed_number AS bed_number,
  b.floor AS floor,
  b.bed_type AS bed_type,
  b.has_ventilator AS has_ventilator,
  w.name AS ward_name,
  d.name AS department_name
FROM ((beds b
JOIN wards w on((b.ward_id = w.ward_id)))
JOIN departments d on((w.department_id = d.department_id)))
WHERE (b.status = 'AVAILABLE')
```

7.2 `available_doctors_view`

Purpose	AVAILABLE doctors below their workload limit, with department. Not used by the API.
Tables	`doctors`, `departments`
Updatable	YES (UPDATE through one base table at a time; no INSERT through a join)

```
SELECT doc.doctor_id AS doctor_id,
  doc.name AS doctor_name,
  doc.specialization AS specialization,
  doc.phone AS phone,
  doc.current_workload AS current_workload,
  doc.max_workload AS max_workload,
  dep.name AS department_name
FROM (doctors doc
JOIN departments dep on((doc.department_id = dep.department_id)))
WHERE ((doc.status = 'AVAILABLE') and (doc.current_workload < doc.max_workload))
```

7.3 `bed_status_summary_view`

Purpose	Bed counts per ward, department, bed type and status. Not used by the API (the bed dashboard runs its own GROUP BY, query 9).
Tables	`beds`, `wards`, `departments`
Updatable	NO

```
SELECT w.name AS ward_name,
  d.name AS department_name,
  b.bed_type AS bed_type,
  b.status AS status,count(b.bed_id) AS count
FROM ((beds b
JOIN wards w on((b.ward_id = w.ward_id)))
JOIN departments d on((w.department_id = d.department_id)))
GROUP BY w.name,d.name,b.bed_type,b.status
```

7.4 `current_admissions_view`

Purpose	ACTIVE admissions with patient, doctor, bed, ward and department names. Not used by the API.
Tables	`admissions`, `patients`, `doctors`, `beds`, `wards`, `departments`
Updatable	YES (UPDATE through one base table at a time; no INSERT through a join)

```
SELECT a.admission_id AS admission_id,
  a.admission_date AS admission_date,
  a.diagnosis AS diagnosis,
  p.name AS patient_name,
  p.patient_id AS patient_id,
  doc.name AS doctor_name,
  doc.doctor_id AS doctor_id,
  b.bed_number AS bed_number,
  b.bed_type AS bed_type,
  w.name AS ward_name,
  dep.name AS department_name
FROM (((((admissions a
JOIN patients p on((a.patient_id = p.patient_id)))
JOIN doctors doc on((a.doctor_id = doc.doctor_id)))
JOIN beds b on((a.bed_id = b.bed_id)))
JOIN wards w on((b.ward_id = w.ward_id)))
JOIN departments dep on((a.department_id = dep.department_id)))
WHERE (a.status = 'ACTIVE')
```

7.5 `doctor_workload_view`

Purpose	Every doctor with department, status, shift, workload and remaining capacity (max – current). Not used by the API.
Tables	`doctors`, `departments`
Updatable	YES (UPDATE through one base table at a time; no INSERT through a join)

```
SELECT doc.doctor_id AS doctor_id,
  doc.name AS name,
  doc.specialization AS specialization,
  dep.name AS department,
  doc.status AS status,
  doc.shift AS shift,
  doc.current_workload AS current_workload,
  doc.max_workload AS max_workload,(doc.max_workload - doc.current_workload) AS available_capacity
FROM (doctors doc
JOIN departments dep on((doc.department_id = dep.department_id)))
```

7.6 `emergency_queue_view`

Purpose	Open emergency cases (WAITING, TRIAGED) ordered by severity and arrival time, with patient and ambulance. Used by the stored procedure sp_get_emergency_queue; the API builds its own queue query.
Tables	`emergency_cases`, `patients`, `ambulances`
Updatable	NO

```
SELECT e.emergency_id AS emergency_id,
  e.severity AS severity,
  e.arrival_time AS arrival_time,
  e.symptoms AS symptoms,
  e.required_specialization AS required_specialization,
  e.required_bed_type AS required_bed_type,
  e.ventilator_required AS ventilator_required,
  p.name AS patient_name,
  a.vehicle_number AS ambulance_number
FROM ((emergency_cases e
LEFT JOIN patients p on((e.patient_id = p.patient_id)))
LEFT JOIN ambulances a on((e.ambulance_id = a.ambulance_id)))
WHERE (e.status in ('WAITING','TRIAGED'))
ORDER BY (case e.severity when 'CRITICAL' then 1 when 'VERY_SERIOUS' then 2 when 'SERIOUS' then 3 when
'MODERATE' then 4 when 'STABLE' then 5 end),e.arrival_time
```

7.7 hospital_occupancy_view

Purpose	Per bed type: total, occupied, available and unavailable beds, and the occupancy rate (%). Not used by the API.
Tables	`beds`
Updatable	NO

```
SELECT beds.bed_type AS bed_type,count(0) AS total_beds,sum((case when (beds.status = 'OCCUPIED') then 1
else 0 end)) AS occupied_beds,sum((case when (beds.status = 'AVAILABLE') then 1 else 0 end)) AS
available_beds,sum((case when (beds.status in ('MAINTENANCE','RESERVED')) then 1 else 0 end)) AS
unavailable_beds,round(((sum((case when (beds.status = 'OCCUPIED') then 1 else 0 end)) / count(0)) *
100),2) AS occupancy_rate
FROM beds
GROUP BY beds.bed_type
```

7.8 pending_bills_view

Purpose	PENDING and PARTIAL bills with patient name, phone and balance due. Not used by the API.
Tables	`bills`, `patients`
Updatable	YES (UPDATE through one base table at a time; no INSERT through a join)

```
SELECT b.bill_id AS bill_id,
  b.bill_date AS bill_date,
  p.name AS patient_name,
  p.phone AS phone,
  b.total_amount AS total_amount,
  b.paid_amount AS paid_amount,(b.total_amount - b.paid_amount) AS balance_due,
  b.status AS status
FROM (bills b
JOIN patients p on((b.patient_id = p.patient_id)))
WHERE (b.status in ('PENDING','PARTIAL'))
```

7.9 v_current_stock

Purpose	Per medicine: the stored total next to the batch total, usable (unexpired) and expired quantity, and the next expiry. Added with batches (migration 006) and **used by the API** (GET /api/medicines, query 8). It is also how a reader can see the stored total and the batch total side by side.
Tables	`medicines`, `medicine_batches`
Updatable	NO

```
SELECT m.medicine_id AS medicine_id,
  m.name AS name,
  m.stock_quantity AS stock_quantity,coalesce(sum(b.quantity),0) AS batch_total,coalesce(sum((case when
(b.expiry_date >= curdate()) then b.quantity end)),0) AS usable_quantity,coalesce(sum((case when
```

```
(b.expiry_date < curdate()) then b.quantity end)),0) AS expired_quantity,min((case when ((b.quantity >
0) and (b.expiry_date >= curdate())) then b.expiry_date end)) AS next_expiry
FROM (medicines m
LEFT JOIN medicine_batches b on((b.medicine_id = m.medicine_id)))
GROUP BY m.medicine_id,m.name,m.stock_quantity
```

8. Triggers

There are seven triggers. All are row-level (FOR EACH ROW) and run inside the statement that fires them, so they are part of the same transaction: if the trigger fails, the statement fails and rolls back.

8.1 `after_admission_discharge`

Timing, event	AFTER UPDATE ON `admissions`
What it does	Only on ACTIVE → DISCHARGED: frees the bed (AVAILABLE), subtracts 1 from the doctor's workload, and closes the open bed_assignment_log row with the discharge time.
Why a trigger	The reverse of the insert trigger, for the same reason. It fires only on that exact transition, so re-saving a discharged admission changes nothing.
Without it	Beds would never become free and workloads would only grow, so the allocator would run out of beds and doctors. *Known gap:* an admission *inserted* already DISCHARGED fires the insert trigger but never this one, leaving the bed OCCUPIED. The seed data did exactly that, and migrations 004 and 005 repaired it.

```
BEGIN
    IF OLD.status = 'ACTIVE' AND NEW.status = 'DISCHARGED' THEN
        UPDATE beds SET status = 'AVAILABLE' WHERE bed_id = OLD.bed_id;
        UPDATE doctors SET current_workload = current_workload - 1 WHERE doctor_id = OLD.doctor_id;

        UPDATE bed_assignment_log
        SET released_at = NEW.discharge_date, status = 'RELEASED'
        WHERE admission_id = NEW.admission_id AND status = 'ACTIVE';
    END IF;
END
```

8.2 `after_admission_insert`

Timing, event	AFTER INSERT ON `admissions`
What it does	Marks the bed OCCUPIED, adds 1 to the admitting doctor's current_workload, and opens a bed_assignment_log row (ACTIVE) for the admission.
Why a trigger	Whoever creates an admission (the API, sp_admit_patient, sp_allocate_emergency, or an administrator at the SQL prompt), the bed, the workload and the history change in the same statement. No caller can forget a step.
Without it	Without it, an admitted bed would stay AVAILABLE and could be given to a second patient. Workload would stop limiting a doctor's admissions, and the bed history would have gaps.

```
BEGIN
    UPDATE beds SET status = 'OCCUPIED' WHERE bed_id = NEW.bed_id;
    UPDATE doctors SET current_workload = current_workload + 1 WHERE doctor_id = NEW.doctor_id;

    INSERT INTO bed_assignment_log (bed_id, patient_id, admission_id, assigned_at, status)
    VALUES (NEW.bed_id, NEW.patient_id, NEW.admission_id, NEW.admission_date, 'ACTIVE');
END
```

8.3 `audit_logs_no_delete`

Timing, event	BEFORE DELETE ON `audit_logs`
What it does	Rejects every DELETE on audit_logs.
Why a trigger	As above. Foreign-key cascades do not fire triggers, so the schema also avoids cascades into audit_logs. The FK to users is RESTRICT (users are deactivated, never deleted), and patient_id has no FK at all.
Without it	Audit rows could be removed, for example to hide a read of a patient record.

```
BEGIN
  SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'audit_logs is append-only: rows cannot be deleted';
```

```
END
```

8.4 audit_logs_no_update

Timing, event	BEFORE UPDATE ON `audit_logs`
What it does	Rejects every UPDATE on audit_logs (SIGNAL 45000, "audit_logs is append-only: rows cannot be updated").
Why a trigger	Makes the audit trail append-only for everyone, including the application. A bug, an attacker using the app's credentials, or a careless admin query cannot rewrite history. The app account has no TRIGGER, DROP or ALTER privilege, so it cannot remove the trigger either.
Without it	Audit rows could be edited, which defeats the purpose of an audit trail.

```
BEGIN
  SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'audit_logs is append-only: rows cannot be updated';
END
```

8.5 after_bed_assignment_log

Timing, event	AFTER INSERT ON `bed_assignment_log`
What it does	**Nothing**: its body is empty (only a comment, "just for demonstration").
Why a trigger	Left from the original project.
Without it	Nothing. It is dead code: it costs a trigger invocation on every bed assignment and should be dropped.

```
BEGIN
    -- This trigger is just for demonstration if we want to cascade logs or notify systems.
    -- The actual logging is handled in the after_admission_insert and update triggers.
END
```

8.6 before_medicine_stock_update

Timing, event	BEFORE UPDATE ON `medicines`
What it does	Rejects any UPDATE that would make medicines.stock_quantity negative (SIGNAL SQLSTATE 45000, "Medicine stock cannot be negative").
Why a trigger	A last line of defence that holds for every writer. CHECK (stock_quantity >= 0) enforces the same rule (MySQL enforces CHECK since 8.0.16), so this trigger is a second guard with a clearer message.
Without it	Nothing on its own, because the CHECK remains. Remove both and an oversell would go through silently.

```
BEGIN
    IF NEW.stock_quantity < 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Error: Medicine stock cannot be negative.';
    END IF;
END
```

8.7 after_prescription_dispense

Timing, event	AFTER UPDATE ON `prescription_items`
What it does	When a prescription item changes from dispensed = FALSE to TRUE, subtracts its quantity from medicines.stock_quantity.
Why a trigger	Ties the stock total to the act of dispensing at the row level, in the same statement as the dispense UPDATE.
Without it	Without it, dispensed medicine would stay in stock_quantity and the total would no longer equal the sum of the batches (the dispense service takes the same quantity off the batches). If the trigger were removed, the service would have to decrement the total itself.

```
BEGIN
    IF OLD.dispensed = FALSE AND NEW.dispensed = TRUE THEN
```

```
        UPDATE medicines
        SET stock_quantity = stock_quantity - NEW.quantity
        WHERE medicine_id = NEW.medicine_id;
    END IF;
END
```

9. Stored procedures and functions

There are **five stored procedures and no stored functions** (`information_schema.ROUTINES` lists only procedures). All five come from the original project, run as `SQL SECURITY DEFINER` with definer `root@localhost`, and are **not called by the API**. The API implements these operations in its services, with locking, auditing and the visit lifecycle. The app account has EXECUTE, so it could call them.

9.1 `sp_admit_patient`

Parameters: `IN p_patient_id INT, IN p_doctor_id INT, IN p_bed_id INT, IN p_department_id INT, IN p_diagnosis TEXT`

1. INSERT one admissions row with admission_date = NOW() and status ACTIVE.
2. The after_admission_insert trigger then occupies the bed, raises the workload and opens the bed log.

> No checks of its own: an occupied bed or a doctor at their limit is not detected here, and CHECK doctors_chk_1 stops only the workload overflow. Not called by the API, whose admission code locks and checks first.

9.2 `sp_allocate_emergency`

Parameters: `IN p_emergency_id INT`

1. Declare an EXIT HANDLER FOR SQLEXCEPTION that rolls back and signals "Emergency allocation failed".
2. START TRANSACTION.
3. Read the case FOR UPDATE, if it is WAITING or TRIAGED. If its patient_id is NULL, roll back with "not found or not waiting". (An arrival not yet registered is therefore refused.)
4. Pick the lowest-id AVAILABLE bed of the required type (any type when none is required) FOR UPDATE SKIP LOCKED, so concurrent allocations skip beds another transaction is taking. None: roll back.
5. Pick the AVAILABLE doctor of the required specialization with the lowest workload below the limit, FOR UPDATE SKIP LOCKED. None: roll back.
6. INSERT the admission (department = the bed's ward's department; the triggers fire).
7. UPDATE the case to ADMITTED with the doctor and bed, then COMMIT.

> The API does not call it. services/emergencyAllocationService.js implements the same idea in application code with plain FOR UPDATE, a doctor fallback, and the creation or linking of an EMERGENCY visit, which this procedure cannot do.

9.3 `sp_discharge_patient`

Parameters: `IN p_admission_id INT`

1. UPDATE the admission to DISCHARGED with discharge_date = NOW(), only if it is ACTIVE.
2. The after_admission_discharge trigger frees the bed, lowers the workload and closes the bed log.

> Does not cancel pending ward doses. The API discharge does (it cancels PENDING medication_administrations in the same transaction).

9.4 `sp_generate_bill`

Parameters: `IN p_admission_id INT`

1. Read the admission's patient_id.
2. INSERT a PENDING bill.
3. INSERT one fixed line: "Base Consultation Fee", CONSULTATION, 1 × 500.00.
4. Set bills.total_amount to the SUM of its items.

> No explicit transaction or locking, and a fixed fee: a demonstration procedure. Not called by the API.

9.5 `sp_get_emergency_queue`

Parameters: `(none)`

1. SELECT * FROM emergency_queue_view.

> A wrapper around the view. Not called by the API.

10. Complex SQL queries

Fifteen real queries:

- twelve from the application and its test helpers (the file is named under each query);
- two from `database/complex_queries.sql`, the original project's query collection;
- one locking sequence assembled from the dispense service.

Parameters are shown as ?, with the value used noted under each query. The samples come from running each query on `hospital_db_test` (MySQL 9.7.1), in a session set to `READ ONLY`. The two locking queries (14 and 15) cannot run in a read-only transaction, so they ran in an ordinary transaction that contained only those SELECTs and was rolled back: nothing was written.

10.1 Is this patient in the doctor's care set? (access control)

Technique: UNION in a derived table · **Source:** server/utils/patientAccess.js (isInDoctorCareSet)

```
SELECT 1 AS in_care_set FROM (
  SELECT patient_id FROM appointments WHERE doctor_id = ?
  UNION SELECT patient_id FROM admissions WHERE doctor_id = ?
  UNION SELECT patient_id FROM consultations WHERE doctor_id = ?
  UNION SELECT patient_id FROM lab_orders WHERE doctor_id = ?
  UNION SELECT patient_id FROM prescriptions WHERE doctor_id = ?
  UNION SELECT patient_id FROM encounters WHERE doctor_id = ?
) AS care_set WHERE patient_id = ? LIMIT 1
```

Parameters: `1`, `1`, `1`, `1`, `1`, `1`, `1`

What it answers. Is the patient in doctor 1's care set? A doctor and a patient are connected through any of six tables; UNION merges them and removes duplicates. Every request a doctor makes for a patient's chart runs this first: no row means 403.

Sample result (hospital_db_test):

in_care_set
1

1 row.

10.2 A patient's prescriptions, each with its labelled medicine lines

Technique: Correlated subquery with GROUP_CONCAT · **Source:** server/controllers/patientController.js (getHistory)

```
SELECT p.prescription_id, p.status,
  (SELECT GROUP_CONCAT(CONCAT(m.name, ' · Dose: ', pi.dosage, ' · Frequency: ', pi.frequency,
          ' · Duration: ', pi.duration, ' · Quantity: ', pi.quantity) SEPARATOR '\n')
   FROM prescription_items pi JOIN medicines m ON pi.medicine_id = m.medicine_id
   WHERE pi.prescription_id = p.prescription_id) AS medication_details
FROM prescriptions p
WHERE p.patient_id = ?
ORDER BY p.prescription_date DESC
```

Parameters: `1`

What it answers. All prescriptions of one patient, newest first, each with its medicine lines as one labelled string. The correlated subquery runs once per outer row and joins that prescription's items to the medicine names.

Sample result (hospital_db_test):

prescription_id	status	medication_details
8	CREATED	Ranitidine 150mg · Dose: 1 tablet · Frequency: Once daily · Duratio… Ciprofloxacin 500mg · Dose: 500 mg · Frequency: Twice daily · Durat…

prescription_id	status	medication_details
9	CREATED	Ranitidine 150mg · Dose: 2 tablet · Frequency: At night · Duration:…
7	DISPENSED	Ranitidine 150mg · Dose: 1 tablet · Frequency: Once daily · Duratio…
4	CREATED	Ranitidine 150mg · Dose: 1 tab · Frequency: Once daily · Duration: …

4 rows.

10.3 One day's appointments with patient, doctor and visit status

Technique: Multi-table LEFT JOIN · **Source:** server/controllers/appointmentController.js (list, ?date=)

```
SELECT a.appointment_id, a.appointment_time, a.status, p.name AS patient_name,
       d.name AS doctor_name, e.encounter_id, e.status AS encounter_status
FROM appointments a
LEFT JOIN patients p ON a.patient_id = p.patient_id
LEFT JOIN doctors d ON a.doctor_id = d.doctor_id
LEFT JOIN encounters e ON e.appointment_id = a.appointment_id
WHERE 1=1 AND a.appointment_date = ?
ORDER BY a.appointment_date, a.appointment_time, a.appointment_id
```

Parameters: `2026-10-11`

What it answers. The appointments of one day with patient and doctor names and the linked visit's status. LEFT JOIN keeps appointments that have no visit yet (encounter columns NULL).

Sample result (hospital_db_test):

appointment_id	appointment_time	status	patient_name	doctor_name	encounter_id	encounter_status
1	09:00:00	BOOKED	Aarav Sharma	Dr. John Smith	*NULL*	*NULL*
5	09:30:00	COMPLETED	Rohan Mehta	Dr. Lakshmi Reddy	2	FINISHED
2	10:00:00	BOOKED	Diya Patel	Dr. Vikram Sharma	*NULL*	*NULL*
3	10:30:00	CHECKED_IN	Ananya Gupta	Dr. Priya Patel	1	ARRIVED
4	11:00:00	BOOKED	Kavya Joshi	Dr. John Smith	*NULL*	*NULL*

15 rows, first 5 shown.

10.4 Visits left open from earlier days

Technique: LEFT JOIN with OR across both tables · **Source:** server/controllers/appointmentController.js (list, ?open_before=)

```
SELECT a.appointment_id, a.appointment_date, a.status, e.status AS encounter_status
FROM appointments a
LEFT JOIN encounters e ON e.appointment_id = a.appointment_id
WHERE a.appointment_date < ?
  AND (a.status IN ('CHECKED_IN', 'IN_PROGRESS') OR e.status IN ('ARRIVED', 'TRIAGED', 'IN_PROGRESS'))
ORDER BY a.appointment_date, a.appointment_time, a.appointment_id
```

Parameters: `2026-10-12`

What it answers. Appointments from before today that are still checked in or whose visit is still open: the "Still open from earlier days" list. The OR condition spans both tables, so it is written over the LEFT JOIN.

Sample result (hospital_db_test):

appointment_id	appointment_date	status	encounter_status
10	2026-10-08	CHECKED_IN	ARRIVED

appointment_id	appointment_date	status	encounter_status
3	2026-10-11	CHECKED_IN	ARRIVED

2 rows.

10.5 A nurse's wards: every bed, and its patient if occupied

Technique: LEFT JOIN chain + IN (subquery) · **Source:** server/controllers/nurseController.js (station)

```
SELECT w.name AS ward_name, b.bed_number, b.status AS bed_status,
       p.name AS patient_name, d.name AS doctor_name
FROM beds b
JOIN wards w ON w.ward_id = b.ward_id
LEFT JOIN admissions a ON a.bed_id = b.bed_id AND a.status = 'ACTIVE'
LEFT JOIN patients p ON p.patient_id = a.patient_id
LEFT JOIN doctors d ON d.doctor_id = a.doctor_id
WHERE b.ward_id IN (
  SELECT ward_id FROM nurse_ward_assignments
  WHERE nurse_user_id = ? AND start_date <= CURDATE()
    AND (end_date IS NULL OR end_date >= CURDATE()))
ORDER BY w.name, b.bed_number
```

Parameters: `15`

What it answers. Every bed in the wards nurse1 covers today, with the patient and admitting doctor for occupied beds. The LEFT JOINs keep empty beds in the result; the subquery is the nurse's current ward assignments.

Sample result (hospital_db_test):

ward_name	bed_number	bed_status	patient_name	doctor_name
General Ward B	GEN-B01	AVAILABLE	*NULL*	*NULL*
General Ward B	GEN-B02	AVAILABLE	*NULL*	*NULL*
General Ward B	GEN-B03	AVAILABLE	*NULL*	*NULL*
General Ward B	GEN-B04	OCCUPIED	Suresh Pillai	Dr. Vikram Sharma
General Ward B	GEN-B05	AVAILABLE	*NULL*	*NULL*

14 rows, first 5 shown.

10.6 Open outpatient visits with the time vitals were last taken

Technique: Correlated subquery with MAX · **Source:** server/controllers/nurseController.js (station)

```
SELECT e.encounter_id, e.status, p.name AS patient_name, d.name AS doctor_name,
  (SELECT MAX(v.recorded_at_utc) FROM vital_signs v
   WHERE v.encounter_id = e.encounter_id) AS last_vitals_utc
FROM encounters e
JOIN patients p ON p.patient_id = e.patient_id
JOIN doctors d ON d.doctor_id = e.doctor_id
WHERE e.encounter_type = 'OUTPATIENT' AND e.status IN ('ARRIVED', 'TRIAGED', 'IN_PROGRESS')
ORDER BY FIELD(e.status, 'ARRIVED', 'TRIAGED', 'IN_PROGRESS'), e.arrived_at
```

What it answers. Outpatients waiting or with the doctor, with the time vitals were last recorded on that visit (NULL: not taken yet). The correlated subquery runs once per open visit.

Sample result (hospital_db_test):

encounter_id	status	patient_name	doctor_name	last_vitals_utc
1	ARRIVED	Ananya Gupta	Dr. Priya Patel	*NULL*
7	ARRIVED	Header Bed mv3gitut	Dr. Amit Das	*NULL*
8	ARRIVED	Booking Person mv3giy9q	Dr. John Smith	*NULL*

3 rows.

10.7 Vital-sign readings with where each was filed

Technique: CASE over two optional LEFT JOIN paths · **Source:** server/controllers/vitalsController.js (READING_SQL)

```
SELECT v.vital_id, v.pulse_bpm, v.temperature_c, u.full_name AS recorded_by_name,
  CASE WHEN v.encounter_id IS NOT NULL THEN CONCAT('Visit with ', vd.name)
       WHEN v.admission_id IS NOT NULL THEN CONCAT('Admission · ', aw.name, ' bed ', ab.bed_number)
  END AS filed_under
FROM vital_signs v JOIN users u ON u.user_id = v.recorded_by
LEFT JOIN encounters ve ON ve.encounter_id = v.encounter_id
LEFT JOIN doctors vd ON vd.doctor_id = ve.doctor_id
LEFT JOIN admissions va ON va.admission_id = v.admission_id
LEFT JOIN beds ab ON ab.bed_id = va.bed_id
LEFT JOIN wards aw ON aw.ward_id = ab.ward_id
ORDER BY v.recorded_at_utc DESC, v.vital_id DESC
```

What it answers. Each vitals reading with who recorded it and where it was filed: under a visit (shows the doctor) or under an admission (shows the ward and bed). Only one LEFT JOIN path matches each row, and CASE picks it.

Sample result (hospital_db_test):

vital_id	pulse_bpm	temperature_c	recorded_by_name	filed_under
6	81	*NULL*	Sunita Devi	Admission · ICU Ward bed ICU-04
5	77	*NULL*	Sunita Devi	Visit with Dr. John Smith
4	88	37.9	Sunita Devi	Visit with Dr. John Smith
3	84	*NULL*	Sunita Devi	Admission · ICU Ward bed VICU-02
2	92	37.4	Sunita Devi	Visit with Dr. John Smith

6 rows, first 5 shown.

10.8 Medicine stock picture (uses the view v_current_stock)

Technique: View in use + correlated subquery · **Source:** server/routes/medicineRoutes.js (GET /api/medicines)

```
SELECT m.medicine_id, m.name, m.stock_quantity, v.usable_quantity, v.expired_quantity, v.next_expiry,
  (SELECT COALESCE(SUM(b.quantity), 0) FROM medicine_batches b
   WHERE b.medicine_id = m.medicine_id
     AND b.expiry_date BETWEEN CURDATE() AND CURDATE() + INTERVAL 90 DAY) AS expiring_soon_quantity
FROM medicines m JOIN v_current_stock v ON v.medicine_id = m.medicine_id
ORDER BY m.medicine_id
```

What it answers. The stock picture per medicine: stored total, usable (unexpired) and expired quantity and next expiry from the view v_current_stock, plus the quantity expiring within 90 days.

Sample result (hospital_db_test):

medicine_id	name	stock_quantity	usable_quantity	expired_quantity	next_expiry	expiring_soon_quantity
1	Paracetamol 500mg	982	982	0	2027-12-31	0
2	Amoxicillin 250mg	500	500	0	2027-06-30	0
3	Ibuprofen 400mg	794	794	0	2027-09-30	0
4	Omeprazole 20mg	295	295	0	2028-01-15	0
5	Aspirin 75mg	1500	1500	0	2027-08-01	0

22 rows, first 5 shown.

10.9 Bed counts by type and status

Technique: Aggregate with GROUP BY · **Source:** server/controllers/bedController.js (stats)

```
SELECT bed_type, status, COUNT(*) AS count
FROM beds
GROUP BY bed_type, status
```

What it answers. How many beds of each type are in each status (the bed dashboard).

Sample result (hospital_db_test):

bed_type	status	count
GENERAL	AVAILABLE	17
ICU	AVAILABLE	1
NICU	AVAILABLE	2
ISOLATION	AVAILABLE	5
VENTILATOR_ICU	AVAILABLE	1

12 rows, first 5 shown.

10.10 Medicines whose stored total disagrees with their batches (invariant check)

Technique: LEFT JOIN + GROUP BY + HAVING · **Source:** server/tests/helpers.js (stockMismatches), run after every stock test

```
SELECT m.medicine_id, m.stock_quantity, COALESCE(SUM(b.quantity), 0) AS batch_total
FROM medicines m LEFT JOIN medicine_batches b ON b.medicine_id = m.medicine_id
GROUP BY m.medicine_id, m.stock_quantity
HAVING m.stock_quantity <> COALESCE(SUM(b.quantity), 0)
```

What it answers. Medicines whose stored total disagrees with the sum of their batches. HAVING filters groups after aggregation; an empty result (as here) means the invariant holds. The tests run it after every stock operation.

Sample result (hospital_db_test):

```
(no rows)
```

10.11 Patients with more than one active admission (anomaly check)

Technique: GROUP BY + HAVING · **Source:** database/complex_queries.sql (query 4)

```
SELECT patient_id, COUNT(*) AS active_admissions
FROM admissions
WHERE status = 'ACTIVE'
GROUP BY patient_id
HAVING COUNT(*) > 1
```

What it answers. Patients with more than one ACTIVE admission at the same time, which the schema cannot forbid by itself (section 3, 12). Empty means the rule holds.

Sample result (hospital_db_test):

```
(no rows)
```

10.12 Departments ranked by number of admissions

Technique: Aggregate + window function RANK() · **Source:** database/complex_queries.sql (query 9)

```
SELECT d.name, COUNT(a.admission_id) AS admission_count,
       RANK() OVER (ORDER BY COUNT(a.admission_id) DESC) AS dept_rank
FROM departments d
```

```
LEFT JOIN admissions a ON d.department_id = a.department_id
GROUP BY d.department_id
```

What it answers. Departments ranked by number of admissions. RANK() is computed after GROUP BY, over the grouped rows; ties share a rank and the next rank is skipped. Departments with no admissions get 0 (LEFT JOIN).

Sample result (hospital_db_test):

name	admission_count	dept_rank
Emergency	8	1
ICU	4	2
Cardiology	3	3
General Medicine	3	3
Pediatrics	2	5

9 rows, first 5 shown.

10.13 Distinct patients a doctor saw today (dashboard)

Technique: UNION of three sources in a derived table + COUNT(DISTINCT) · **Source:** server/controllers/doctorController.js (getTodayAnalytics)

```
SELECT COUNT(DISTINCT patient_id) AS patients_seen FROM (
  SELECT patient_id FROM appointments WHERE doctor_id = ? AND appointment_date = CURDATE()
  UNION
  SELECT patient_id FROM emergency_cases WHERE assigned_doctor_id = ?
    AND arrival_time >= CURDATE() AND arrival_time < CURDATE() + INTERVAL 1 DAY
    AND status IN ('ADMITTED', 'DISCHARGED')
  UNION
  SELECT patient_id FROM admissions WHERE doctor_id = ?
    AND discharge_date >= CURDATE() AND discharge_date < CURDATE() + INTERVAL 1 DAY
    AND status = 'DISCHARGED'
) AS t
```

Parameters: `1`, `1`, `1`

What it answers. How many distinct patients doctor 1 dealt with today across appointments, emergency cases and discharges. UNION removes duplicates before the count.

Sample result (hospital_db_test):

patients_seen
1

1 row.

10.14 Dispensing: lock the prescription, then medicines in id order, then batches first-expiry-first

Technique: SELECT ... FOR UPDATE (three locking reads in one transaction) · **Source:** server/services/dispenseService.js, server/utils/medicineLocks.js, server/utils/batches.js

```
SELECT status, patient_id FROM prescriptions WHERE prescription_id = ? FOR UPDATE
```

```
SELECT medicine_id, stock_quantity FROM medicines WHERE medicine_id = ? FOR UPDATE   -- once per
medicine, ascending id
```

```
SELECT batch_id, batch_number, quantity, expiry_date FROM medicine_batches
WHERE medicine_id = ? AND quantity > 0 AND expiry_date >= CURDATE()
ORDER BY expiry_date, batch_id FOR UPDATE
```

Parameters: `1`, `3`, `3`

What it answers. The locking reads of a dispense, in lock order. Locking the prescription first serialises any second dispense. The medicine rows come next, one at a time in ascending id. Then the unexpired batches, earliest expiry first, through idx_medicine_batches_fefo. The sample shows the rows each step locked.

Sample result (hospital_db_test):

Step 1:

status	patient_id
CREATED	9

1 row.

Step 2:

medicine_id	stock_quantity
3	794

1 row.

Step 3:

batch_id	batch_number	quantity	expiry_date
3	LEGACY-3	794	2027-09-30

1 row.

10.15 Bedside dose: lock one scheduled dose with its order, prescription and medicine

Technique: Four-table JOIN with SELECT ... FOR UPDATE · **Source:** server/controllers/marController.js (lockDose)

```
SELECT ma.administration_id, ma.status, ma.patient_id, ma.scheduled_at_utc,
       pi.dosage, pi.route, pi.dispensed, p.status AS prescription_status, m.name AS medicine_name
FROM medication_administrations ma
JOIN prescription_items pi ON pi.item_id = ma.prescription_item_id
JOIN prescriptions p ON p.prescription_id = pi.prescription_id
JOIN medicines m ON m.medicine_id = pi.medicine_id
WHERE ma.administration_id = ? FOR UPDATE
```

Parameters: `1`

What it answers. Locks one scheduled dose together with its order, prescription and medicine. FOR UPDATE on a join locks the matching rows of every joined table, so a second nurse waits, then sees the dose is no longer PENDING.

Sample result (hospital_db_test):

administration_id	status	patient_id	scheduled_at_utc	dosage	route	dispensed	prescription_status	medicine_name
1	PENDING	6	2026-10-11 08:30:00	500mg	ORAL	1	DISPENSED	Paracetamol 500mg

1 row.

11. Transactions and concurrency

11.1 ACID as applied here

- **Atomicity.** Every write path runs in one transaction (`withTransaction`: BEGIN … COMMIT, or ROLLBACK on any error). The audit row for a data change is written *inside* that transaction, so either the change and its audit row are both stored, or neither is; a test makes the audit insert fail and checks the change was rolled back. A dispense with one short item changes nothing at all: no stock, no batches, no items marked.
- **Consistency.** Constraints (74 FKs, 29 CHECKs, 16 unique keys, ENUMs, NOT NULL, strict SQL mode), triggers (beds, workload, stock, audit) and the invariants the services keep (stock total = sum of batches, appointment status = visit status). Errors return a clear 4xx before anything is written.
- **Isolation.** InnoDB at REPEATABLE READ (the server default; the app does not change it). Plain reads see a consistent snapshot. Every read that a later write depends on is a *locking* read (FOR UPDATE or FOR SHARE), so it sees the latest committed row and holds it until commit. Pessimistic locking, not version numbers, is the pattern throughout.
- **Durability.** InnoDB's redo log; the binary log is on (log_bin = 1); migrate.js backs up before every migration; and the hosted demo takes nightly dumps, keeping 7 (restore tested).

11.2 Pessimistic locking and lock order

A deadlock needs two transactions waiting on each other's locks in a cycle. Every multi-row write path here locks rows in one fixed global order, so no cycle can form:

Operation	Locks, in this order
Dispense	prescriptions row (FOR UPDATE) → medicines rows, deduplicated and in **ascending medicine_id** (FOR UPDATE) → each medicine's batches through idx_medicine_batches_fefo (expiry_date, batch_id; FOR UPDATE) → prescription_items by primary key (UPDATE)
Prescribe	medicines rows in ascending medicine_id (**FOR SHARE**), up front, before inserting the items
Stock receipt / adjustment	medicines row → its batches (same order as dispensing)
Visit transitions	appointments row → encounters row; starting a visit also locks the patients row (FOR UPDATE), so two starts for one patient are serialised
Emergency allocation	emergency case → bed → doctor → that doctor's open visits with the patient (by primary key)
Bedside dose	the dose row joined to its order, prescription and medicine (FOR UPDATE)
Orders filed under a visit	the in-progress encounter or the active admission (FOR SHARE), so it cannot close while the order is written

11.3 The dispensing deadlock (fixed)

Before. The original dispense locked the medicines rows through a join, in the order the items happened to be stored. Two prescriptions for the same pair of medicines in opposite order, [A, B] and [B, A], each locked one medicine and waited for the other. InnoDB detected the cycle and killed one with ER_LOCK_DEADLOCK (1213).

Fix. The fix has three parts:

1. Total the quantity per medicine, which also merges two lines of the same medicine.
2. Lock the medicines one row at a time in ascending medicine_id (`server/utils/medicineLocks.js`).
3. Only then check stock and lock batches.

The same bug from another angle. Inserting a prescription_items row takes a *shared* lock on the referenced medicines row (the foreign-key check). A prescription created at the same moment as a dispense took those shared locks in item order, which could still cycle with the dispense's exclusive locks. Prescribing now takes FOR SHARE locks on its medicines up front, in the same ascending order.

Evidence. Tests with 30 concurrent rounds each (opposite item order; prescribing during dispensing; stock receipts during dispensing) and an opt-in stress test (300 rounds × 2 scenarios) record 0 deadlocks and stock that stays consistent.

11.4 The gap-lock issue (fixed)

Under REPEATABLE READ, a locking read through a secondary index takes *next-key* locks: the matching index records plus the gaps before them, which block inserts into those gaps.

Before. The dispense read its items with a locking read:

```
SELECT ... FROM prescription_items WHERE prescription_id = ? FOR UPDATE
```

That took next-key locks on the `prescription_id` index *before* taking the medicines locks. A concurrent prescription create already held medicines locks and then needed to insert items into that same index range, so it waited on the gap lock while the dispense waited on its medicines lock: a cycle.

Fix (`server/services/dispenseService.js`):

- The items are read with a *plain* (non-locking) read. The prescriptions row lock already serialises every dispenser of that prescription, and nothing else modifies its items.
- Items are marked dispensed with an UPDATE *by primary key* (`WHERE item_id IN (…) AND dispensed = FALSE`), which locks only those records, not gaps.

The emergency allocator has the same kind of fix. It finds the doctor's open visits with a plain read, then locks them by primary key in id order. A locking read through the (patient, status) index would lock the index entry before the row, while a concurrent "start visit" holds the row and then updates that index entry.

11.5 Preventing double actions

Race	Prevented by	Proven by
Double dispensing	Both requests lock the prescriptions row FOR UPDATE, so the second waits. When it continues, it sees status DISPENSED and returns 409. Besides that, the item UPDATE has `AND dispensed = FALSE` and the code checks that affectedRows equals the number of items.	Test "double dispense: two concurrent requests for one prescription — exactly one succeeds"
Double booking	`UNIQUE (doctor_id, appointment_date, appointment_time)`: the second insert fails with ER_DUP_ENTRY and the API answers 409 "Double booking detected". The unique index check is atomic, so no lock or read-then-insert race is involved. *Limitation:* only identical start times collide; overlapping slots (09:00 and 09:10) are not detected.	Test "booking the same doctor slot twice returns 409"
Double dose administration	The dose row is locked FOR UPDATE with its order, and its status must still be PENDING, so the second nurse gets 409 "already ADMINISTERED". `UNIQUE (prescription_item_id, scheduled_at_utc)` stops a dose from being scheduled twice in the first place.	Test "two nurses giving the same dose at once: exactly one succeeds"
Double visit start / check-in	The appointment row lock with a state check (an invalid transition returns 409). `UNIQUE (appointment_id)` on encounters. The patients row lock serialises two starts for one patient.	Tests "racing start and cancel …", "two visits … started at the same moment: exactly one starts"

If a deadlock still happens (InnoDB error 1213), the transaction is rolled back. The API answers with an error body carrying `code: "DB_DEADLOCK"` and `retryable: true` (HTTP 500), so a client knows the request is safe to retry.

12. Integrity and security

12.1 Entity integrity

Every table has a primary key: an AUTO_INCREMENT integer surrogate, or `version` for schema_migrations. Primary key columns are NOT NULL by definition. Natural identifiers that must be unique carry unique keys:

- username and email (users);
- a department's name;
- an ambulance's vehicle_number;
- a bed code within its ward;
- a batch number within its medicine;
- an ABHA number (NULLs allowed);
- a stored file name.

12.2 Referential integrity

74 foreign keys; every ON UPDATE rule is NO ACTION (InnoDB treats it as RESTRICT). Primary keys are surrogates and are never updated, so cascading updates are not needed. ON DELETE rules:

- **RESTRICT (54)** wherever the child has a life of its own, e.g. a patient with visits cannot be deleted.
- **CASCADE (12)** where the child is part of the parent: bill_items of a bill, prescription_items of a prescription, the result of an order, attachments of a result, a doctor's schedules and analytics.
- **SET NULL (8)** for optional references whose loss should not delete the child: an emergency's ambulance, assigned doctor and assigned bed; an admission's emergency case; a note's appointment; a lab order's note; a lab test's department; the technician on a result.

Some CASCADEs are questionable in a clinical system, and a few rules are inconsistent:

- Deleting an admission would delete its bills (`bills_ibfk_2`) and its consultations (`consultations_ibfk_4`).
- Deleting a patient or doctor would delete their surgery requests (`surgery_requests_ibfk_1/2`).
- A consultation's prescriptions cascade (`prescriptions_ibfk_1`), while its lab orders are SET NULL.

In practice nothing deletes these rows: the API has no DELETE routes at all (records are cancelled or deactivated). The rules are still part of the schema, and listed in section 3.

12.3 Domain integrity

- **Types:** DATE and TIME for clinical dates, DECIMAL for money and lab values, and small integers for vitals.
- **ENUMs** for every status and code (e.g. appointments.status, beds.bed_type, prescription_items.route).
- **29 CHECK constraints**, for example:
  - impossible vitals (pulse 20–250, SpO2 50–100, temperature 25.0–45.0 °C, systolic above diastolic, both or neither BP value);
  - a vital reading filed under a visit or an admission;
  - dates in order (discharge after admission, an assignment's end after its start);
  - non-negative stock and money;
  - paid ≤ total;
  - an ABHA number of exactly 14 digits;
  - an ADMINISTERED dose has a time.
- **SQL mode** includes STRICT_TRANS_TABLES, so an out-of-range value or a bad ENUM value is an error, not a silent truncation.
- **The API** adds the rules a schema cannot express: a date of birth not in the future, a dose that includes a unit, and the visit-state machine.

12.4 Role-based access

- **Roles.** Six roles live in `users.role`. A JWT carries the role, and every route declares the roles allowed (`authorize(...)`).
- **Which patients.** `server/utils/patientAccess.js` decides patient by patient:
  - a doctor sees their *care set* (query 1: a UNION over appointments, admissions, consultations, lab orders, prescriptions and encounters);
  - a nurse sees active admissions in the wards assigned today, plus open outpatient visits;
  - front desk and admin see demographics but not clinical notes;
  - lab and pharmacy see their own work.
- **Audit.** Denied requests are answered 403 and audited (outcome DENIED).

12.5 The restricted database user

The application connects as `hospital_app`, which has exactly these grants:

```
GRANT USAGE ON *.* TO hospital_app
GRANT SELECT, INSERT, UPDATE, DELETE, EXECUTE ON hospital_db.* TO hospital_app   (and the test/demo
databases)
```

- It cannot CREATE, ALTER or DROP anything, create or drop triggers, GRANT, LOCK TABLES, read or write files, or touch another database.
- Triggers, views and procedures run with their definer's rights, so they keep working.
- The server refuses to start as root.
- Migrations, backups and the schema diff use a separate admin account (`DB_ADMIN_USER`), only for those tools.
- On the hosted demo, the web server process does not even have the admin credentials in its environment.

12.6 The immutable audit log

- `audit_logs` is append-only: BEFORE UPDATE and BEFORE DELETE triggers reject every change (section 8).
- Data changes are audited in the same transaction as the change; reads and denials are written by middleware after the response.
- `details` holds ids, enum values and changed field names only, never passwords, tokens or note text. A test plants a marker string in a note and checks it never reaches the log.
- The foreign key to users is RESTRICT, and accounts are deactivated rather than deleted.
- patient_id deliberately has no foreign key, so no patient change can cascade into the trail.

12.7 The migration system

- Numbered files in `database/migrations` (000–014) are applied in order by `server/scripts/migrate.js` and recorded in `schema_migrations`.
- 000 is a schema-only snapshot of the live database; on an existing database it is only marked as applied.
- Before applying anything to a database that has tables, migrate.js takes a mysqldump backup.
- Migrations are *additive* (new tables, nullable columns, guarded DDL), so they are safe to re-run.
- Each one was rehearsed on a copy of live data first.
- `scripts/schema-diff.js` compares hospital_db with the test database (tables, columns, indexes, FKs, checks, triggers, routines, views, events) and must report no differences.

13. Evolution

The original project (captured as migration 000, a snapshot of the live database before the roadmap) had 25 tables, 8 views, 5 stored procedures and 5 triggers:

`admissions, ambulances, appointments, bed_assignment_log, beds, bill_items, bills, consultations, departments, doctor_daily_analytics, doctor_schedules, doctors, emergency_cases, lab_orders, lab_result_attachments, lab_results, lab_tests, medicines, patients, pharmacy_stock_movements, prescription_items, prescriptions, surgery_requests, users, wards`

The roadmap added 7 tables, 1 view and 2 triggers, plus `schema_migrations`, and extended existing tables with nullable columns only:

Migration	Feature	Schema change
001	Audit trail	New `audit_logs`; triggers `audit_logs_no_update`, `audit_logs_no_delete`
002	Visits (encounter lifecycle)	New `encounters`; `encounter_id` on consultations, lab_orders, prescriptions, surgery_requests (backfilled)
003	Nurse ward assignments; allergies	New `nurse_ward_assignments`; `patients.allergies`
004, 005	Data fixes	Beds left OCCUPIED with no admission set back to AVAILABLE (no schema change)
006	Batches and FEFO dispensing	New `medicine_batches`, `prescription_item_batches`; `pharmacy_stock_movements.batch_id`; view `v_current_stock`; one LEGACY batch per medicine
007	Medication administration record	New `medication_administrations`; structured order columns on prescription_items
008	Demo data	A dispensed ward prescription so the MAR is not empty (through the dispense service)
009	Lab reference ranges	Numeric bounds on lab_tests and lab_results; `numeric_value`, `interpretation_source`
010	ABHA number	`patients.abha_number` (unique, 14-digit CHECK); phone index
011	Vitals flowsheet	New `vital_signs` with 11 CHECKs
012	Emergency visits	An EMERGENCY encounter for each active emergency admission (data only)
013	Orders filed under a visit or admission	`admission_id` on lab_orders and prescriptions
014	Estimated date of birth	`patients.dob_estimated`

The roadmap also added composite indexes for the new queries, and changed the application's use of the original tables without changing their structure:

- the deadlock-free dispense;
- the appointment status driven by the visit;
- the restricted database account;
- access rules per role and per patient.

14. Viva questions

Forty likely questions with short model answers, roughly in the order of this report. The last ones are about weaknesses and trade-offs.

1. **How many tables, views, triggers and procedures does the database have?**
   33 tables (32 for the hospital, plus schema_migrations), 9 views, 7 triggers, 5 stored procedures, no stored functions and no events. There are 74 foreign keys, 29 CHECK constraints and 126 indexes (33 primary, 16 unique, 77 other).
2. **Why surrogate integer keys instead of natural keys?**
   Natural identifiers are missing (most patients have no ABHA number), change (phone, name) or are composite. A small, immutable integer keeps foreign keys narrow and makes InnoDB's clustered index compact. Natural identifiers that must be unique get unique keys instead.
3. **What is the difference between appointments and encounters?**
   An appointment is a booking (a slot). An encounter is the visit itself, with its own lifecycle and timestamps, and can exist without an appointment (walk-in, emergency, inpatient). An appointment has at most one encounter (unique appointment_id), and the appointment's status mirrors the visit's.
4. **Is the schema in 3NF?**
   Most tables are in BCNF. Several are deliberately not in 3NF: they copy patient_id or doctor_id that is also reachable through a visit or admission, and some store derived values (stock total, bill total, workload, bed status, daily analytics). Section 6 lists each case and what keeps it consistent.
5. **Give an example of a partial dependency in this schema.**
   medication_administrations has the candidate key (prescription_item_id, scheduled_at_utc), but patient_id depends on prescription_item_id alone. That fails 2NF with respect to that key. The value is copied when the dose is created and never changes.
6. **Why keep medicines.stock_quantity when batches already hold the stock?**
   It makes "how much of X do we have" a single-row read and gives the CHECK and trigger one place to stop overselling. The cost is an update anomaly risk. That risk is controlled by changing both in one transaction under the same locks, by the dispense trigger, and by an invariant query that finds any mismatch (query 10).
7. **What would you change about medicines.expiry_date?**
   Drop it, or turn it into a view column (MIN of the unexpired batch expiries). Since batches were added, it is not maintained and can mislead.
8. **How is a double booking prevented?**
   UNIQUE (doctor_id, appointment_date, appointment_time). The second insert fails atomically with ER_DUP_ENTRY, and the API returns 409. Weakness: only identical start times collide, so overlapping appointments are not detected.
9. **How is double dispensing prevented?**
   The dispense locks the prescriptions row FOR UPDATE, so a second dispense waits. When it resumes it sees status DISPENSED and returns 409. The item update also requires dispensed = FALSE and checks the affected-row count. A test sends two dispenses at once and exactly one succeeds.
10. **What causes a deadlock, and how did you remove the dispensing one?**
    Two transactions each holding a lock the other needs. Dispenses locked medicines in item order, so [A,B] and [B,A] formed a cycle. Every path now locks medicines one by one in ascending medicine_id (FOR UPDATE to dispense, FOR SHARE to prescribe), so no cycle can form. Stress tests show 0 deadlocks over hundreds of rounds.
11. **What is a gap lock, and where did it bite?**
    Under REPEATABLE READ, a locking read on a secondary index also locks the gaps between index records, blocking inserts there. The dispense's FOR UPDATE read of prescription_items by prescription_id locked that index range before the medicines rows, and a concurrent prescription create needed to insert there while holding medicines locks. The fix is a plain read (the prescription row lock already serialises dispensers) and an update by primary key.

12. **Why not use SERIALIZABLE isolation instead of explicit locks?**
    SERIALIZABLE turns every plain SELECT into a shared locking read. That would make deadlocks more likely and slow every read. Explicit FOR UPDATE / FOR SHARE on the few rows a decision depends on gives the same safety where it matters, with a lock order we control.
13. **What does FOR UPDATE SKIP LOCKED do in sp_allocate_emergency?**
    It skips rows locked by another transaction instead of waiting. Two concurrent allocations therefore take different beds and doctors immediately rather than queueing. The trade-off is that the "best" bed may be skipped. The API's allocator uses plain FOR UPDATE instead.
14. **Why are some rules triggers and others application code?**
    Triggers hold for every client (API, stored procedures, admin SQL) and run in the same statement: bed occupancy, workload, stock decrement, audit immutability. Rules that need context the database does not have (the user's role, the patient's assignment, the visit state machine, the Five Rights at the bedside) live in the application.
15. **What is the weakness of the admission triggers?**
    They act only on INSERT (occupy) and on the exact UPDATE ACTIVE → DISCHARGED (release). An admission inserted already DISCHARGED occupies a bed forever. The seed data did this, and migrations 004/005 repaired it. A TRANSFERRED status is not handled at all.
16. **Why is there a trigger that does nothing?**
    after_bed_assignment_log has an empty body, left from the original project as a placeholder. It should be dropped.
17. **How is the audit log made tamper-resistant?**
    BEFORE UPDATE and BEFORE DELETE triggers raise an error. The app account cannot drop triggers. Nothing cascades into the table, because the users FK is RESTRICT and patient_id has no FK. Data changes are audited in the same transaction, and details never hold secrets or note text.
18. **Can root still alter the audit log?**
    Yes: an account with TRIGGER and DROP privileges (root, or the admin account used for migrations) can drop the triggers and then change rows. Database-level immutability protects against the application and its credentials, not against the DBA. For that you would copy the log to write-once storage or another system, and review the admin account's use.
19. **Why does audit_logs.patient_id have no foreign key?**
    A foreign key would either block patient changes (RESTRICT) or cascade into the audit table, and cascades do not fire the immutability triggers. The audit trail must outlive any patient-record change, so it stores the id as a plain indexed value.
20. **What privileges does the application account have?**
    SELECT, INSERT, UPDATE, DELETE and EXECUTE on its own database only: no DDL, TRIGGER, GRANT, FILE or LOCK TABLES. A SQL-injection or a stolen password cannot change the schema or remove the audit triggers.
21. **How are migrations made safe?**
    Each is additive and guarded (IF NOT EXISTS, information_schema checks), so a failed run can be repeated. migrate.js backs up first and records versions in schema_migrations. Each migration is rehearsed on a copy of live data, and a schema diff against the test database must be empty.
22. **Which relationships are many-to-many and how are they implemented?**
    Prescriptions and medicines (prescription_items); prescription lines and batches (prescription_item_batches); nurses and wards over time (nurse_ward_assignments). lab_orders is an associative entity between patients, doctors and lab_tests, and bed_assignment_log one between beds and admissions over time.
23. **Which relationships are one-to-one?**
    lab_orders → lab_results (unique order_id) and appointments → encounters (unique appointment_id), both 1 to 0..1. users → doctors is meant to be 1:1, but doctors.user_id is not unique, so the schema allows 1:N: a weakness.
24. **Why are some foreign keys ON DELETE CASCADE?**
    Where the child cannot exist without the parent: bill_items, prescription_items, a lab result, attachments. Some CASCADEs are questionable for clinical data (deleting an admission would delete its bills and consultations). They are safe only because nothing deletes those rows.

25. **What is the redundant index in this schema?**
    appointments has UNIQUE (doctor_id, appointment_date, appointment_time) and also idx_appointments_doc_date_time on the same columns. The second adds write cost and storage for no benefit.
26. **Why does idx_patients_name not help the patient search?**
    The search uses name LIKE '%text%'. A leading wildcard cannot use a B-tree index, so MySQL scans. Options: a FULLTEXT index, a prefix search, or a separate search column.
27. **Explain idx_medicine_batches_fefo.**
    (medicine_id, expiry_date, batch_id) matches "WHERE medicine_id = ? ORDER BY expiry_date, batch_id FOR UPDATE". The rows are found and locked in FEFO order without a sort, and every writer locks batches in that same index order, which prevents batch-lock deadlocks.
28. **Which views are updatable?**
    MySQL reports available_beds_view, available_doctors_view, current_admissions_view, doctor_workload_view and pending_bills_view as updatable (one base table per UPDATE, no inserts through joins). The aggregate views and emergency_queue_view are not. The application writes only to base tables.
29. **Are the stored procedures used?**
    Not by the API. They come from the original project. The API implements admission, allocation, discharge and billing in services with explicit locking, auditing and the visit lifecycle, which the procedures do not have.
30. **What does ACID atomicity mean concretely here?**
    A dispense either marks every item dispensed, takes stock off the medicine total and batches, writes the batch trace, the ledger, the scheduled doses and the audit row, or does none of it. A test with one item short of stock checks that nothing changed.
31. **How do you keep appointment and visit status consistent under concurrency?**
    One function performs every transition. It locks the appointment row then the encounter row, validates the transition against a fixed table, and writes both in the same transaction. Tests race a start against a cancel and check that exactly one wins and the mirror holds.
32. **How would you find data that violates an invariant?**
    With GROUP BY … HAVING queries, e.g. query 10 (stock total ≠ sum of batches) and query 11 (patients with more than one active admission). An empty result means the invariant holds.
33. **What is the role of CHECK constraints vs ENUMs?**
    ENUMs restrict a column to a fixed set (statuses, routes). CHECKs express ranges and cross-column rules (systolic > diastolic, paid ≤ total, end ≥ start). Both are enforced by MySQL 8.0.16+ for every writer. Before 8.0.16, CHECKs were parsed and ignored.
34. **Why store times in UTC in medication_administrations?**
    Doses are scheduled and compared across days and shifts. Storing UTC (*_at_utc) avoids ambiguity from the server's time zone; the app converts to IST only for display.
35. **What is the cost of copying patient_id into many tables?**
    Update anomaly risk (a row's patient could disagree with its visit's patient) and more storage. The benefit: direct filtering and access checks by patient without joining through optional parents. It is controlled by resolving the parent for the given patient under a lock, and never updating the copy.
36. **How would you prevent overlapping appointments, not only identical ones?**
    Store a slot end time and check for overlap inside a transaction that locks the doctor's day, for example by locking the doctors row FOR UPDATE before checking. A plain unique key cannot express "no overlap" in MySQL.
37. **How is the "one active admission per patient" rule enforced?**
    Not by the schema: MySQL has no partial unique index (UNIQUE WHERE status = 'ACTIVE'). The application checks it while holding the relevant row locks, and query 11 detects any violation. A generated column holding patient_id only when ACTIVE, with a unique index on it, would enforce it in the database.

38. **Is JSON in audit_logs.details a normalization problem?**
    Strictly, it breaks 1NF, because a structured value sits in one column. It is accepted because details are never joined or filtered on. The indexed, queryable facts (user, action, entity, patient, outcome, time) are proper columns, and details holds only supplementary ids and field names.

39. **What are the biggest weaknesses of this database design?**
    Copied patient_ids and stored totals that rely on application discipline; doctors.user_id not unique; a redundant index; questionable CASCADEs; a stale medicines.expiry_date; free-text dose and frequency next to structured columns; triggers that miss some status paths; an empty trigger; unused original views and procedures.

40. **If you had one more migration, what would it contain?**
    A unique key on doctors.user_id. Drop idx_appointments_doc_date_time and the empty trigger. Change bills→admissions and consultations→admissions to RESTRICT. Make bill_items.total_price a generated column. Add a generated-column unique key for one active admission per patient. Drop medicines.expiry_date, or replace it with a view column.